MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Logical Reasoning for Small-Scale Language Models

LICENSE THESIS

Graduate: **Debora SIKÓ**

Supervisor: **Prof. Dr. Ing. Rodica POTOLEA**
Prof. Dr. Ing. Camelia LEMNARU
As. Drd. Ing. Vlad Andrei NEGRU

2025

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

HEAD OF DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Graduate: **Debora SIKÓ**

Logical Reasoning for Small-Scale Language Models

1. **Project proposal:** This thesis investigates the logical reasoning capabilities of small-scale text-generative language models. It aims to analyze whether small language models can go beyond surface-level understanding, if they are capable of human-like reasoning or they mainly rely on artifacts in the dataset. Therefore, the thesis also includes analysis on a widely used dataset.
2. **Project contents:** Introduction - Project Context, Project Objectives, Bibliographic Research, Analysis and Theoretical Foundation, Detailed Design and Implementation, Testing and Validation, User's Manual, Conclusions, Bibliography
3. **Place of documentation:** Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:**
5. **Data of issue of the proposal:** November 1, 2024
6. **Data of delivery :** September 8, 2025

Graduate: _____

Supervisor: _____

MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Declarație pe proprie răspundere privind autenticitatea lucrării de licență

Subsemnatul(a) _____, _____ legitimat(ă) cu _____ seria _____ nr. _____ CNP _____, autorul lucrării _____ _____ elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Specializarea _____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____ a anului universitar _____, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*.

Data _____

Nume, Prenume _____

Semnătura

Contents

Chapter 1 Introduction - Project Context **1**

Chapter 2 Project Objectives **3**
2.1 Objective 1: Analysis on NLI of smaller text-generative models 3
2.2 Objective 2: Analysis of a dataset for NLI 3
2.3 Objective 3: Fine-tuning a model on NLI and comparison with baseline . 4
2.4 Objective 4: Dataset augmentation with synonyms 5

Chapter 3 Bibliographic Research **6**
3.1 Mathematical reasoning in LLMs 6
3.2 Chain-of-thought reasoning and prompting 9
3.3 Regarding NLI 10

Chapter 4 Analysis and Theoretical Foundation **13**
4.1 NLI foundation 13
4.2 Datasets and Benchmarks for NLI 13
4.2.1 SNLI 13
4.2.2 MultiNLI 14
4.2.3 SICK dataset 15
4.3 Transformer-based models 15
4.3.1 Architecture of transformer networks 15
4.3.2 Encoder-decoder models 17
4.3.3 Encoder-only models 18
4.3.4 Decoder-only models 18
4.4 Training vs Fine-tuning 18
4.4.1 Parameter-efficient Fine-tuning 21
4.5 Dataset manipulation – Wordnet, POS tagging 23
4.5.1 Part-Of-Speech Tagging 23
4.5.2 WordNet 23
4.6 Jaccard Similarity and Distance 24
4.7 Cosine similarity 24

Chapter 5 Detailed Design and Implementation **25**
5.1 General approach 25
5.2 Analysis on the dataset 25
5.3 SICK dataset – augmentation 26
5.3.1 Improved synonym substitution 29
5.4 Models from LM Studio 33
5.4.1 Models 34
5.4.2 API used for testing 34
5.5 Fine-tuning with LoRa 35
5.6 Converting Fine-tuned model to GGUF format 37

Chapter 6 Testing and Validation **38**
6.1 Dataset analysis results 38
6.1.1 Sentence length analysis 38
6.1.2 Jaccard Similarity 39
6.2 Gemma vs gemma fine-tuned results 40
6.3 Dataset augmentation results 41
6.4 Gemma results for augmented dataset 42
6.4.1 Augmented dataset results on Gemma 3 and Gemma 3 fine-tuned 42
6.5 Results for different prompts on Gemma 43
6.6 NLI results for different models compared 44
6.7 Heatmap of Gemma 45

Chapter 7 User's Manual **48**
7.1 Local environment 48
7.2 LM Studio models 49
7.3 Google Colab and Kaggle 51
7.4 fine-tuned model replication 51

Chapter 8 Conclusions **53**
8.1 Personal Contributions and Critical Result Analysis 53
8.1.1 Analysis on NLI of smaller text-generative models 53
8.1.2 Dataset Analysis 53
8.1.3 Fine-tuning and Evaluation 54
8.1.4 Dataset Augmentation and Interpretation of Results 54
8.2 Possibilities of Future Improvements 55

Bibliography **57**

Chapter 1. Introduction - Project Context

The rapidly increasing field of Natural Language Processing (NLP) is an intersection domain of artificial intelligence (AI) and machine learning, which focuses on the way certain machines interpret human language or some form of it. The language is interpreted by Large Language Models (LLMs), which are systems capable of emulating human-like understanding and, in many cases, generating text that closely resembles our natural language. Some important subdomains of NLP are now used in areas ranging from medicine and law to everyday applications such as casual chatbots. Practical applications of NLP include sentiment analysis, toxicity classification, named entity recognition, machine translation, spam detection, question answering, and text generation.

Two main branches of this domain are Natural Language Understanding (NLU), which focuses on the comprehension and interpretation of human language, and Natural Language Generation (NLG), which focuses on producing coherent and contextually relevant text.

As LLMs become increasingly widespread in many NLP applications, one question becomes more and more important: do they really "think"? The question might sound trivial or even naive, since the mathematical functions and algorithms of LLMs are well-defined in the literature. However, it is important to investigate and analyze their "thought process," meaning whether they are truly capable of conventional reasoning in the logical sense. This investigation becomes increasingly urgent as the usage of LLMs grows across both professional and casual domains. This thesis aims to analyze the logical reasoning capabilities of smaller LLMs.

As the baseline of this analysis, one particular task stands out: Natural Language Inference (NLI). This seemingly simple task is one of the main building blocks of our logical system. The main idea of NLI is to understand and define the logical relationship between two consecutive sentences — determining whether the second sentence is entailed by, contradicts, or is neutral with respect to the first. It serves as a base metric to measure whether a model understands the meaning behind the words rather than merely predicting text based on surface-level patterns.

In this research, the focus falls on smaller LLMs (with a maximum of 1b parameters). The reason for this is twofold: first, smaller models are more accessible for research and deployment in resource-constrained environments; second, they are increasingly used in real-world applications where running large-scale models is impractical. Understanding their reasoning capabilities is essential to ensure safe and reliable use in contexts where decision-making or logical interpretation is critical.

While in NLG tasks, the main usage of transformer based text-generative models — for example, in open-ended text generation — logical reasoning may not be strictly necessary to produce fluent and grammatically correct text, it is nevertheless valuable to evaluate these models on reasoning-focused metrics. This becomes especially relevant when such models are, or will be, deployed in domains where accuracy, interpretability,

and logical consistency are essential, such as healthcare decision support, legal document analysis, or educational tools.

Considering the structure of this thesis, first we present the main project objectives and aims, relevant to the project. Secondly, the literature surrounding LLMs, reasoning in NLP and NLI are introduced in the Bibliographic Research chapter.In the Analysis and Theoretical foundations chapter main notions, model architectures and approaches are explained, to give a basis to the project. In the Detailed Design and Implementation chapter, the code is described for analyzing the models and the dataset, which is followed by the Testing and Validation chapter, which contains the performance results of the LLMs on the experiments conducted, as well as the analysis of the dataset. The thesis is concluded with potential directions for future work.

Chapter 2. Project Objectives

The research presented in this thesis focuses on evaluating and enhancing the reasoning capabilities of small-scale Large Language Models (LLMs) in the context of NLI. This work aims to bridge the gap between the impressive text-generation performance of modern LLMs and their ability to perform logical reasoning tasks that require a deeper semantic understanding of language.

With the growing accessibility of transformer-based models, there is an increasing demand for deploying them in real-world applications, including resource-constrained environments.

2.1. Objective 1: Analysis on NLI of smaller text-generative models

Large language models with tens or hundreds of billions of parameters currently dominate state-of-the-art benchmarks across natural language tasks. However, these models are computationally expensive to train and deploy, making it impractical for real-life applications, especially in resource constrained environments. Smaller models — in the range of less than 1–7 billion parameters — present a good balance between capability, cost, and accessibility. However, their reasoning abilities remain less explored. The main goal of this objective is to get a better insight into the capabilities of these smaller models in logic reasoning, especially on the task NLI. We mainly employ generative models, composed of stacked layers of decoders.

The following subgoals are defined for this objective:

- **Selection of the models:**
  The main focus being small, transformer-based models with less than (or equal to) 1 billion parameters.
- **Accuracy on NLI:**
  A comparative analysis will be conducted between the models, on the task of NLI.
- **Prompting:**
  Since the main focus of this thesis is smaller text-generative models, it is useful to look at how a model performs when given different versions of the same prompt. In these versions, the wording of the prompt may change, but the main instruction stays the same. It is important, because it evaluates the robustness of the model. The evaluation should be performed on the same dataset for each version of the prompt.

2.2. Objective 2: Analysis of a dataset for NLI

Before we analyze the models on NLI, it is important to investigate whether the metric by which the measurements are done is artifact free. Therefore, the goal of this

objective is to provide a baseline analysis on the dataset, which will be used in the experiments.

We will check if there are patterns in the data that might make the task easier without real reasoning, such as the volume of overlapping between sentence pairs, which might give away the answer. Understanding the dataset quality will help us trust our results and make fair comparisons between models.

This objective contains the following subgoals:

- **Analysis of word count across labels:**
  The purpose of this objective is to analyze whether there exists an artifact due to the large difference in sentence length between labels.
- **Analysis of word count, premise-hypothesis difference:**
  The purpose of this sub-objective is to analyze whether the difference in length between the premise sentence and the hypothesis sentence indicates an artifact. If one specific label contains mostly sentence pairs where the hypothesis is significantly shorter than the premise, this suggests a bias in the data. If this pattern is not observed across other labels, it indicates the presence of an artifact.
- **Lexical similarity:**
  This objective aims to analyze how similar the two sentences of the premise-hypothesis pairs are in terms of phrasing. It is an important analysis, because high lexical overlap in the two sentences, when it is present in one label in particular, also indicates an artifact.

2.3. Objective 3: Fine-tuning a model on NLI and comparison with baseline

To gain a better insight into the true potential of a model, we will additionally fine-tune the pre-trained models on NLI benchmarks to assess their reasoning capabilities. The goal of this objective is to conduct such a fine-tuning, and measure the potential improvements compared to the base model and further analyze the model's flaws in logical reasoning.

We will compare the performance before and after fine-tuning to see how much the model can improve when trained specifically for NLI. This will also help us understand if the model's weaknesses are due to a lack of training data for this task or due to more fundamental limitations.

The subgoals of this objective are listed below:

- **Choosing appropriate fine-tuning option:**
  To choose the best option for fine-tuning, that is suitable for the purpose of this thesis both performance-wise and taking into account resource limitations, a thorough analysis is needed on training and fine-tuning.
- **Fine-tuning:**
  The purpose of this objective is to implement the fine-tuning.
- **Evaluation of fine-tuned model**
  This objective aims to analyze and compare the performance of a fine-tuned large language model compared to the vanilla pre-trained version that was not trained on the specific NLI dataset.

2.4. Objective 4: Dataset augmentation with synonyms

In order to improve the robustness of the model's reasoning capabilities, particularly in handling lexical variation, this objective focuses on augmenting the NLI dataset by introducing synonym substitutions. By replacing certain words or phrases with semantically equivalent alternatives, we aim to reduce potential overfitting to specific lexical patterns and encourage the model to rely more on underlying semantic relationships rather than surface-level cues. This augmentation strategy is expected to improve generalization and provide a more reliable evaluation of reasoning ability.

This objective contains the following subgoals:

- **Hypothesis only dataset:**
  To conduct an evaluation on whether a hypothesis-only bias exist in the model, it is necessary to modify the dataset for this purpose, by eliminating the premises.
- **POS tagging:**
  The main purpose of the Dataset augmentation objective is to test whether a model understands the meaning of a word, or phrase, or it relies on the same wording for the NLI task. To achieve the objective, this thesis aims to perform a synonym substitution of the subjects in the premise sentences. The identification of said subject is needed beforehand.
- **Substitution of word with best synonym in the sentence:**
  The purpose of this objective is to develop a solution which finds a suitable synonym for a given word. This solution has to take into account not only the meaning of the word in the dictionary, but also the context of the sentence, and the meaning of that word in the context.

Chapter 3. Bibliographic Research

With the rise of Large Language Models (LLMs), more and more questions have appeared around whether these state-of-the-art language models are truly capable of human-like mathematical and logical reasoning or if they are simply engaging in clever pattern matching based on probability. This chapter presents a bibliographic overview of the key works in this area.

3.1. Mathematical reasoning in LLMs

One of the uses of LLMs could be math problem solving. However, there exist some limitations of language models within this domain. The article "Enhancing the transformer with explicit relational encoding for math problem solving" [1] explores the transformer-based models capabilities within this domain and proposes a solution for enhancing their performance by modifying the way that the self-attention layer works. This modification consists of the addition of another vector to the already existing embedding, which contains additional information regarding the role of that particular token. This helps the model in building a hierarchy-like relational web between the tokens, which is particularly useful in solving mathematical equations. They achieved this by adding another vector R, besides the already existing query, key and value vectors (how the attention layer works in the transformer models is explained in the next chapter). Further on, this role vector is used to calculate a tensor product which will be learned by each attention head. This tensor product consists of a filler vector – which is like a value vector in standard attention, and the role vector, which encodes the relational role of the given embedding. Instead of computing a simple weighted sum of values, the TP-Attention mechanism binds each filler to its corresponding role via a tensor product shown in picture 3.1:

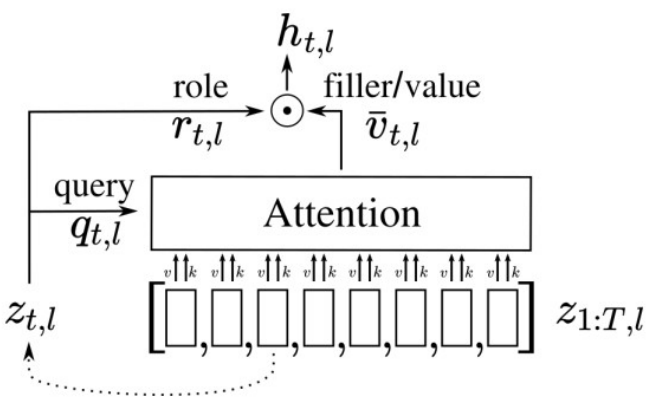

Figure 3.1: TP-Attention mechanism for one head at position t in layer l [1]

At position t in layer l, the current hidden state is $z_{t,l}$ and $z_{1:T,l}$ are all states of layer l. From each $z_{i,l}$ we compute keys k and values v; from $\mathbf{z}_{t,l}$ we compute a query $\mathbf{q}_{t,l}$ and a role vector $\mathbf{r}_{t,l}$. The (single-head) output shown in the figure is the role–filler binding

$$h_{t,l} = r_{t,l} \odot \bar{v}_{t,l},$$

where $\odot$ denotes element-wise (Hadamard) multiplication. The overall output of the TPMHA layer will be:

$$TPMHA(z_{t,l}, z_{1:T,l}) = \sum_h [W^o_{h,l}(\bar{v}^h_{t,l} \odot r^h_{t,l}) + b^o_{h,l}]$$

This explicit role-filler binding allows the model to capture symbolic structure more effectively and enhances its ability to reason over mathematical expressions. The authors show that this method outperforms standard transformers on a dataset of math word problems by enabling better generalization and interpretability of relational structures within equations.

While the authors of the previous paper aimed to improve transformer models to enhance their mathematical reasoning capabilities, the authors of article [2] focused on understanding the limitations of large language models in this domain and introduced a new dataset for this purpose. They argued that the reasoning process in LLMs is closer to probabilistic pattern matching. However, when these models face certain new challenges or when the difficulty level of a problem increases, they fail to provide correct answers with valid proof.

There were four main focus points in which the authors tested 25 state-of-the-art models: the first contribution being the introduction of a new benchmark, the GSM-Symbolic, which based on a symbolic template (shown in figure 3.2) generates variants of GSM8K (Grade School Math 8K) questions. The second area of focus was questioning the reliability of the previously reported GSM8K result. Another contribution was that they demonstrated that there is a different amount of accuracy drop when there is a different kind of change in a question. Finally, they showed that there is a significant decrease in accuracy when we add seemingly important, but in reality, irrelevant information to the problem.

The authors also raised awareness of the fact that GSM8K is the single most popular test set, so there is a great risk of data contamination. In the first part of their work, they generated a new test set based on GSM8K, but they introduced different types of annotations. This step was crucial because it allowed them to separate names from values and other types of words.

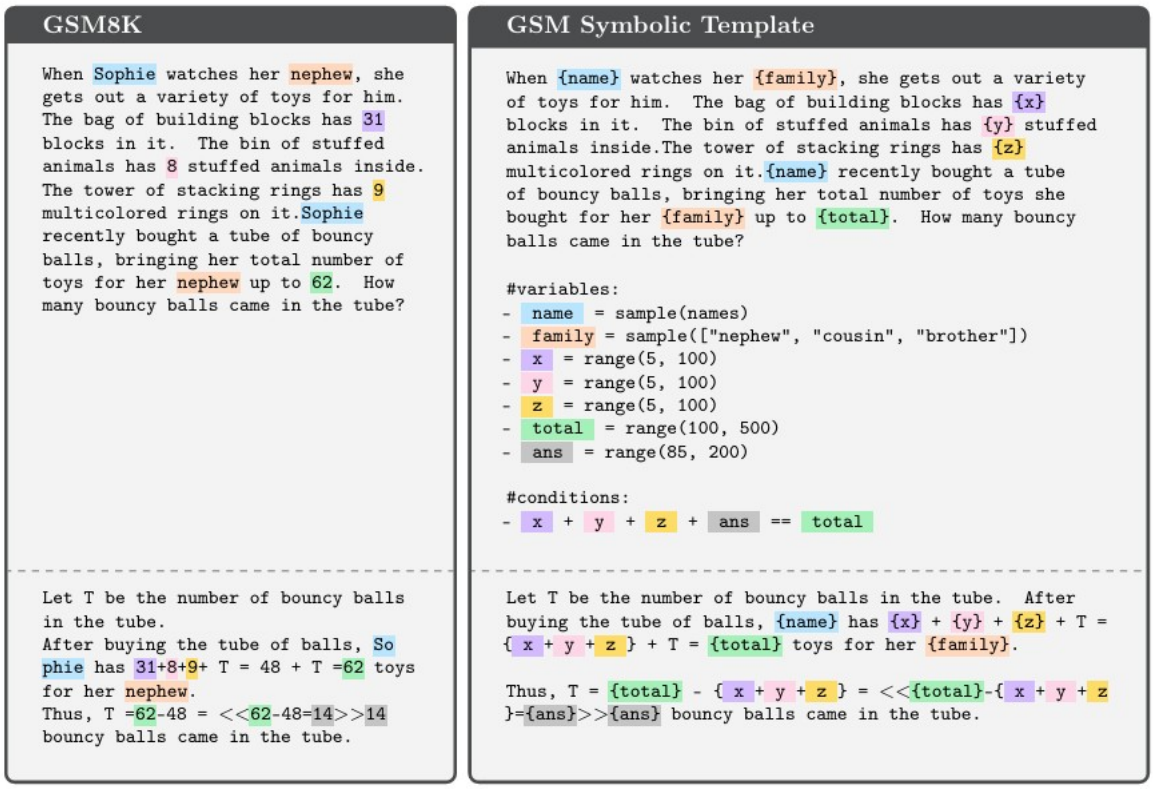

Figure 3.2: GSM symbolic template, from [2]

They showed the accuracy drop between the results for the GSM8K versus the results for GSM-Symbolic. The results varied between -0.3 and -9.2 across different kinds of models. There was also a significant variance in accuracy depending on the type of change made in a problem. LLMs generally achieved much higher accuracy when only a name token was changed in a problem. However, when the value in the math exercise was changed, the accuracy dropped in almost every case. Accuracy was even lower when both a name and a value token were changed in the exercise, highlighting the underlying problem regarding the LLMs' capability to truly reason. Raising the difficulty level also had a similar negative effect on performance. An even more interesting finding was during the introduction of a so-called GSM-NoOp dataset (exemplified by the picture 3.3), which was generated by adding information which was irrelevant to the reasoning and conclusion. The research found that LLMs failed to recognize this and included it in the reasoning of their answer.

GSM-NoOp

Oliver picks 44 kiwis on Friday. Then he picks 58 kiwis on Saturday. On Sunday, he picks double the number of kiwis he did on Friday, but five of them were a bit smaller than average. How many kiwis does Oliver have?

---

o1-mini: Oliver picks kiwis over three days as follows:
- Friday: 44 kiwis
- Saturday: 58 kiwis
- Sunday: Double the number he picked on Friday, which is $2 \times 44 = 88$ kiwis

However, on Sunday, 5 of these kiwis were smaller than average. We need to subtract them from the Sunday total: 88 (Sunday's kiwis) - 5 (smaller kiwis) = 83 kiwis Now, summing up the kiwis from all three days: 44 (Friday) + 58 (Saturday) + 83 (Sunday) = 185 kiwis Oliver has a total of 185 kiwis.

Llama3-8B: Let's break it down step by step:
Oliver picks 44 kiwis on Friday. He picks 58 kiwis on Saturday. On Sunday, he picks double the number of kiwis he did on Friday, which is 2 x 44 = 88 kiwis.
Five of the kiwis picked on Sunday are a bit smaller than average, so we subtract 5 from the total number of kiwis picked on Sunday: 88 - 5 = 83 kiwis. Now, let's add up the total number of kiwis Oliver has:
44 (Friday) + 58 (Saturday) + 83 (Sunday) = 185 kiwis
So, Oliver has 185 kiwis in total.

Figure 3.3: GSM No operation, from [2]

3.2. Chain-of-thought reasoning and prompting

A key question in logical reasoning is whether chain-of-thought prompting actually helps language models find correct answers. In the article [3], the authors showed that using few-shot chain-of-thought examples leads to state-of-the-art results on several math word problem benchmarks, such as GSM8K, and also performs well on symbolic reasoning tasks. Importantly, the authors emphasized that these improvements were only observed in sufficiently large models (>=100b parameters); for smaller models, chain-of-thought prompting either had no effect or even degraded performance by introducing additional confusion, that is the smaller models produced a fluent reasoning, but it was logically incorrect. Another interesting observation was the fact that the performance of this kind of prompting was better the more complicated the problem was.

To test their method, the authors used benchmarks like GSM8K, MultiArith, and CommonsenseQA. They gave the models a few examples that included step-by-step reasoning before the final answer. This helped large models, such as PaLM 540B, break down complex problems and come up with correct and understandable answers. Their findings suggest that this kind of reasoning only starts to work well when models reach a certain size. The authors conclude that although the models seem to produce a human-like thought process, the question whether or not they actually reason still remains unanswered. This unanswered question leads us to the next article, where the authors aimed to lead the model to a correct reasoning process.

In article [4], the authors introduced a new synthetic QA dataset called PRONTOQA with the purpose of analyzing the chain-of-thought reasoning of LLMs. PRONTOQA is the abbreviation for Proof and Ontology-Generated Question-Answering, and it is most closely related to the PROOFWRITER and FOLIO datasets. With these datasets, they introduced some new properties, such as controls for unary versus binary predicates and controls for specific rules of deduction.

The authors analyzed the specific variables that might affect the reasoning of a model. They not only tested the correctness of the given answer but also wanted to check

if the model arrived at a certain conclusion for the right reasons. Their approach consists of four steps:

- Ontology generation from a set of concepts
- Proof generation by traversing the ontology
- Translation of the ontology to natural language
- Translation of the proof to queries and to natural language

In their analysis, it was crucial to divide the steps that a language model makes during the CoT reasoning into categories: strictly-valid atomic correct step (canonical), strictly-valid atomic misleading step, strictly-valid non-atomic correct step, strictly-valid non-atomic misleading step, broadly-valid correct step, broadly-valid misleading step, and invalid step. They found that label accuracy (only focusing on the correctness of the result) is not necessarily a reliable way to measure whether the model is reasoning correctly because a correct answer can result from heuristics. This is highlighted by their results, which show that there is no one-to-one correspondence between label accuracy and "strict proof accuracy" (the metric they used to measure the reasoning ability of LLMs). Another interesting finding was how the size of a model corresponds to its reasoning ability: seemingly, the larger the size of the model, the greater the proof accuracy.

They also raised awareness of the fact that when a model makes an invalid step, the longer it stays in that branch, the less likely it is to draw a correct conclusion in the end.

3.3. Regarding NLI

Natural Language Inference (NLI) is a core task in language understanding and often stands as a baseline for more complex text generation and reasoning tasks. Its importance has led to extensive research not only on improving model performance but also on identifying and mitigating biases present in commonly used datasets. This section of the bibliographic study summarizes key efforts in addressing these issues, including dataset augmentations, bias reduction techniques, and model-level interventions.

Firstly, in [5] a new dataset was introduced for the purpose of analyzing whether in case of a natural language inference task the models arrive to the correct label "the right way", for correct reasons. This dataset includes (beside the premise-hypothesis pairs) additional information in form of explanations of the reasoning for the given label. These explanations were generated via crowdsourcing, where around 6000 annotators were asked to give explanations about the relationship between the premise and hypothesis. There were four types of experiments the authors conducted on models previously trained on e-SNLI, namely: PredictAndExplain, ExplainThenPredict, Represent and Transfer.

In the first one, the model was instructed to predict the correct label, then based on the label provided give an explanation, while in the second one the order was the opposite, the model was first asked to give an explanation of the relationship between the hypothesis and the premise and then based on the explanation to predict the label. The researchers found that when they trained their model on their own e-SNLI dataset, the accuracy of the model was essentially the same as when they trained it on SNLI 4.2.1, however when they evaluated the PredictAndExplain results, the model provided correct explanations with only 34% accuracy. The next experiment aimed to further investigate the explanations given by the model. For this experiment (ExplainThenPredict) there was an aproximate 3% accuracy drop for the correctly predicted label, however there

was an around 20-30% (greater for an attention based model) accuracy increase with the provided explanations.

While the e-SNLI dataset provides some interesting improvements in helping to understand model reasoning, especially for NLI, the correctness of the SNLI dataset still raises some questions, which should be investigated. In the thesis, titled "Mitigating Bias in the SNLI Dataset" [6] the author looks at the problem of bias in the SNLI dataset. It focuses on a specific issue called hypothesis-only bias, where models can guess the correct label using just the hypothesis, without needing to look at the premise. This goes against the goal of Natural Language Inference, which is to understand how two sentences relate to each others.

To fix this, the author creates new examples that are carefully designed to break the patterns that models rely on too easily. These new examples help balance the dataset and make it harder for models to take shortcuts. The idea is to push models to actually understand the meaning of the sentences, not just learn surface patterns.

The authors tests how well different pre-trained models, like DistilBERT, perform when trained on this improved dataset. The results show that models trained on the new version rely less on the hypothesis alone and still make accurate predictions. DistilBERT, in particular, handles the changes well and becomes more resistant to bias.

While the SNLI thesis focuses on reducing dataset bias to improve model fairness and reasoning, the MorphNLI approach tackles the challenge from a different angle—by introducing a more interpretable, stepwise method to guide models toward correct NLI predictions.

In article [7] a new approach is discussed for bettering LLM predictions on the NLI task. The idea of this approach is based on text morphing, and the first step is to generate intermediate steps between the premise and hypothesis sentences. An LLM called trainer is employed to generate morphisms following specific criteria, that the sentences should preserve their meaning, and the changes should not break syntactic boundaries. The incorrect morphisms were filtered out, both the "lazy morphisms" (which refer to nearly no change in the process) and the "short morphisms" (which refer to the fact that the sentences became shorter and shorter than the premise and hypothesis). The morphing process consists of a series of M of such morphisms, where each step is labelled as entailment, neutral or contradiction. The final label is given as follows: if the morphism chain consists only of entailment steps then the relationship between the premise and hypothesis is labelled as entailment, however if any other steps were taken besides entailment, then the final prediction becomes the leftmost step besides the entailment steps. This trainer model was used to further teach a "student" model on NLI, essentially fine-tuning the smaller model on the morphisms generated by the teacher model. In this experiment the authors used models from the GPT family. One of the important results of this experiment was that when evaluated in cross-domain settings (e.g. training on MNLI 4.2.2 and testing on SICK 4.2.3, and vice versa), MorphNLI consistently outperformed strong baseline NLI systems (e.g. BERT model), achieving relative accuracy gains of up to 12.6% . Another advantage of this approach is that it gives a better explainability, a better insight into the reasoning process of the model, due to the explicit morphing steps.

In paper [8], AlphaZero like Tree-Search is discussed, which is a framework known as TS-LLM. It is designed to enhance the reasoning and decision-making capabilities of Large Language Models. Inspired by the success of AlphaZero in games, this uses a learned value function to guide the decoding and training of LLMs. The authors' aim is

to improve the efficiency of LLMs in various tasks.

The most standout feature of TS-LLM is exactly this use of the learned value function. This function evaluates the potential outcomes of different possible actions, to help the model make the best decision choosing the most promising path. This approach makes the model's ability to make more accurate predictions. This framework can be applied to a wide range of tasks, not only to mathematical problems, but also in problems like making strategic decisions in games.

In the domain of mathematical reasoning, TS-LLM was tested on the GSM8K benchmark. The results show that the framework improved the model's ability to solve the mathematical problems. Because of the learned value function, TS-LLM guided the model to explore alternative solution paths, which resulted in increased accuracy and consistency in the math problems. The results of this approach applied to different models is shown on figure 3.4.

They also tested the model on the Game 24 benchmark, which proves difficult even for other state-of-the-art LLMs like GPT-4. In this case, TS-LLM helped the model evaluate many potential moves ahead, similar to strategies used in games like chess.

Task	Policy	Value	Accuracy(%)
GSM8K	GPT-3.5	GPT-3.5 (ToT)	72.7
	GPT-3.5	LLaMA-V (Ours)	**74.0**
	LLaMA-SFT	LLaMA (ToT)	37.4
	LLaMA-SFT	GPT-3.5 (ToT)	45.8
	LLaMA-SFT	LLaMA-V (Ours)	**52.5**
Game24	GPT-3.5	GPT-3.5 (ToT)	15.5
	GPT-3.5	LLaMA-V (Ours)	**19.1**
	LLaMA-SFT	LLaMA (ToT)	9.2
	LLaMA-SFT	GPT-3.5 (ToT)	21.0
	LLaMA-SFT	LLaMA-V (Ours)	**64.8**

Figure 3.4: Results of using AlphaZero like Tree-Search, from [8]

Chapter 4. Analysis and Theoretical Foundation

4.1. NLI foundation

Natural Language Inference (NLI) is a task, in which one determines wether a given „hypothesis" logically follows an also given „premise". In other words, premise can be seen as our definite knowledge about the world, and we have to determine that based on this knowlegde whether the hypothesis is true or not. In Natural Language Processing (NLP) this task can be seen as a classification problem, where the Large Language Model (LLM) has to determine (classify) the label based on the premise and the hypothesis. This label can be of three kinds: contradiction, entailment or neutral. When the hypothesis directly follows the premise logically, it is called an entailment. Contradiction is the case when the hypothesis conflicts with the premise, and the relationship between the two sentences is called Neutral when no definite conclusion can be drawn about the hypothesis, based on the premise alone. The following examples from the SNLI corpus may help to show the essence of the different relationships:

P: 'A person on a horse jumps over a broken down airplane.'
H: 'A person is at a diner, ordering an omelette.'
Label: contradiction

P: 'A person on a horse jumps over a broken down airplane.'
H: 'A person is outdoors, on a horse.'
Label: entailment

P: 'Children smiling and waving at camera'
H: 'They are smiling at their parents'
Label: neutral

This task is particularly important in the analysis of the logical reasoning capabilities of LLMs, since it is a fundamental part of our day-to-day reasoning and language.

4.2. Datasets and Benchmarks for NLI

There are many datasets containing premise-hypothesis pairs for exactly this task – NLI. The most important benchmarks hovewer are the following ones: SNLI, MultiNLI, SuperGLUE, etc.

4.2.1. SNLI

The Stanford Natural Language Inference (SNLI), first introduced in the paper [9], is a widely used benchmark for the NLI task. It contains 570k human-written sentence

pairs in English, which are labeled as contradiction,entailment or neutral manually. The distribution of the labels are: 190113 entailment pairs, 189218 neutral pairs and 189702 pairs are labeled contradiction. During the process of data collection, the authors used Amazon Mechanical Turk for crowdsourcing, where around 2500 participants contributed. Each participant was given the same task, which is to write hypothesis sentences to the premises provided, described by the instruction set in the figure 4.2.

We will show you the caption for a photo. We will not show you the photo. Using only the caption and what you know about the world:

- Write one alternate caption that is **definitely** a **true** description of the photo. *Example: For the caption "Two dogs are running through a field." you could write "There are animals outdoors."*
- Write one alternate caption that **might be** a **true** description of the photo. *Example: For the caption "Two dogs are running through a field." you could write "Some puppies are running to catch a stick."*
- Write one alternate caption that is **definitely** a **false** description of the photo. *Example: For the caption "Two dogs are running through a field." you could write "The pets are sitting on a couch." This is different from the* maybe correct *category because it's impossible for the dogs to be both running and sitting.*

Figure 4.1: Instructions provided for participants in SNLI crowdsourcing [9]

To ensure the correctness of the data, a validation phase was introduced, which covered around 10% of the corpus. Beside the original annotator, an additional four workers labelled the relationship between each sentence, and the final gold label was decided by majority voting. The examples lacked a consensus, were discarded from the dataset. Despite its careful validation, the SNLI dataset still has limitations and biases. One of the more obvious concerns is, that it contains certain lexical cues in the examples, which strongly corelate to one specific label (like negation terms, such as "no" and "nobody", indicate a contradiction label). For example, Gururangan et al. (2018) [10] demonstrated that a simple hypothesis-only classifier achieved 67% accuracy on SNLI, underlining how models might exploit these artifacts rather than performing true inference. Nevertheless, SNLI remains one of the most widely used resources for training and evaluating NLI models.

4.2.2. MultiNLI

The MultiNLI (Multi-Genre Natural Language Inference) [11] dataset follows the same model as SNLI, having in total 433k sentence pairs. It however differs from SNLI, because MNLI has a wide range of genres of written text as well as spoken text. It is also labeled with entailment information.

4.2.3. SICK dataset

The SICK (Sentences Involving Compositional Knowledge) [12] dataset, mainly used in this research, is a collection of 5000 sentence pairs, where besides the usual entailment label, another information is given, which is called the relatedness score. This relatadness score indicates how much the hypothesis relates to the premise in a sense of theme. The labels are human annotated, based on crowdsourcing, just like in case of SNLI. The process of creating the dataset consisted of three main steps sentence normalization, expansion and pairing. During sentence normalization, the sentences sampled from the pre-existing datasets 8K ImageFlickr and SemEval-2012 STS MSR-Video Descriptions, were processed in a way to get rid of named entities and specific dates, to get more general sentences. Then, in the expansion phase three additional sentences were created for each sentence, one with a similar meaning, one that contradicts the original sentence and one that is lexically identical but has different meaning. In the pairing process, the newly generated sentences were paired with the original normalized sentences, that they were derived from, and also with arbitrary sentences as well. In the annotation process, crowdsourcing was utilized, where the annotators rated first how related the two sentences are on a 1 to 5 scale, then the final score was calculated as the average of 10 annotator's score. The annotators also labeled each pair as contradiction, neutral or entailment, likewise, the final gold label was decided by majority voting. The dataset of 10,000 sentence pairs was split evenly into a training set and a test set, with 50% of the data in each.

4.3. Transformer-based models

The transformer model is one kind of neural network, which has excellent results on processing sequential data. The categories of these transformer models are encoder-decoder models, encoder-only models like BERT (Bidirectional Encoder Representation from Transformers), and decoder-only transformer-based models, like Gemma models. These transformer-based models have a main feature, which is their self-attention mechanism. The transformer architecture is defined by the fact that it mainly uses attention layers and basic feedforward layers. Attention mechanism – is the models ability to scan and process a sequence at once and decide about when and how to focus on specific parts of it. Further on, the architecture of a transformer network is presented.

4.3.1. Architecture of transformer networks

On a higher level a transformer network consists of multiple transformer layers, where the input embedding of layer i is the output embedding of layer *i-1*. So the layers are one on top of the other. The number of layers is usually between 2 and 24 for transformer networks.

The internal structure of one individual transformer layer consists of a sequence of five operations. During which each word gets a contextualized embedding of all the words in the sequence. What this means is that in order for the model to understand the meaning of one individual token in the sentence, it has to take into consideration the rest of the sentence too. These five operations aim to achieve just that.

The first operation adds the positional information to each of the input embeddings, that is the position of each word in the given sentence. It achieves this by using hard-coded functions to generate a position vector: for a word i in the input text this

function generates a vector p_i corresponding to that word and its position. Then the input embedding of a word will be the sum of this positional vector and a regular word embedding w_i:

$$x_i = w_i + p_i$$

, where x_i is the input embedding generated. The explanation of the actual function f used to generate this positional vector is beyond the scope of this paper, it is enough to mention that it should be a function which generates unique values for each word in the input sentence. Before transformer networks other types of neural networks used to store the positional information of a word by concatenating it to the input embedding, transformer networks replaced this by adding them together.

The second operation represents the main functionality of the transformer layer, which is the so-called "self attention layer". The intuition behind this operations is that the model learns on which part of the input sequence it should pay more attention in context of each word. The output vector of each word i in this layer will be z_i, these output vectors are computed as follows:

$$z_i = \sum_j a_{ij} v_j$$

, where a_{ij} represents the attention weight between word i and word j and the v_i vector is the value vector, which is the x_i input vector transformed into a different feature space. This way, the output of this layer will be the weighted average of the output embeddings from the first operation. This weighted average comes from the fact that the attention weights are calculated as a probability distribution. Each attention weight is calculated using two other vectors: the q_i query and the k_i key vectors. That is, if we want to get the embedding of one word i we first calculate each attention weight a_{ij} with every corresponding word j. We get this by first calculating the dot product between the query vector of word i and the key vector of word j as follows: $a_{ij} = q_i k_j$. Then to avoid getting to big values the result must be divided by the square root of the size of the vector k which is the same for every vector. That is $a_{ij} = \frac{a_{ij}}{\sqrt{|k_i|}}$.

Finally, the softmax function is applied to every attention weights to get a probability distribution. This step is crucial, so when we add them together, we get a weighted average of the v_i vectors. The vectors q_i, k_i, v_i are generated using the original input vector of this layer x_i by transposing it into a new feature space. These vectors are computed using three matrices W^Q, W^K, W^V, so the vectors are calculated by the multiplication of the input vector x_i with these matrices, respectively. Throughout training these matrices are the parameters that the model should learn. The matrices should have the same amount of rows as the length of the input vector x_i.

Above is the explanation of one self attention layer, but an important technique that is used is called the multi-head attention, which consists of multiple self attention layers. The difference between each self attention layers is that each of them use a different initial value for the matrices W^Q, W^K, W^V. The process of producing output embeddings is repeated at each layer. This helps the model to learn more complex information.

There are three more operations in a transformer layer, which follow the above mentioned self attention mechanism. These operations are the following: addition, normalization and a feed forward mechanism. This is usually divided in two add and normalize layers and a feed forward layer in between.

4.3.2. Encoder-decoder models

In encoder-decoder models the above mentioned layers and operations are put together in the architecture, shown in picture 4.2. The encoder part of the architecture takes the input sequence and converts it "to a sequence of continuous representations z=(z1,...,zn)" ([13], 3). This part is responsible for "understanding" the input, and mapping it into a form the machine can interpret.

Then the decoder part of the architecture takes these embeddings of the encoder, and based on that "understanding" generates an output sequence (y1,...ym). On the left side of the figure the encoder part is visible, while on the right side the decoder is shown.

Both encoder and decoder parts of the architecture use stacked attention layers, along with feed forward and add and normalize layers. The decoder part of the architecture takes information from both the encoder and the already generated output embeddings.

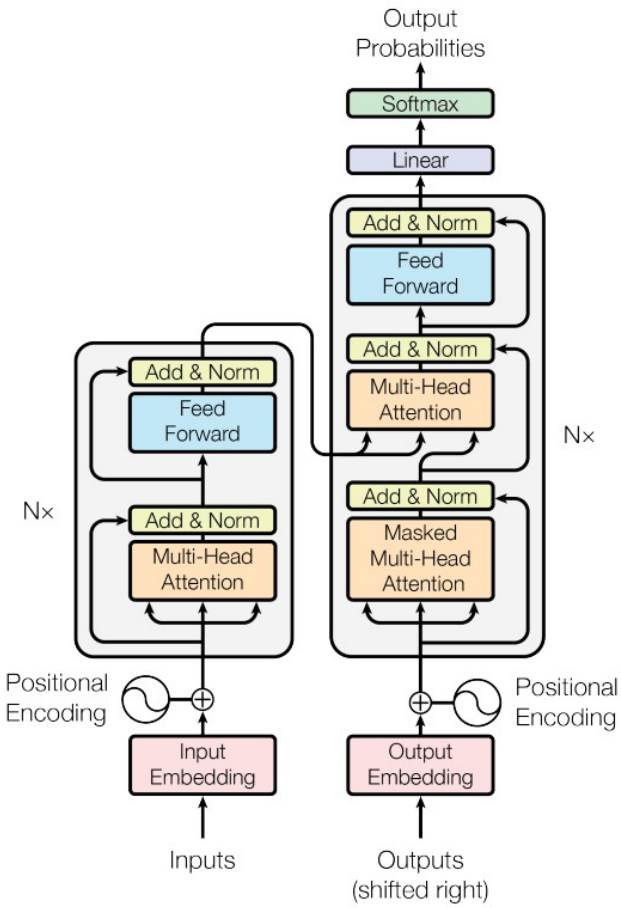

Figure 4.2: Transformer architecture from "Attention is All You Need" paper [13]

An example for this type of model is the T5 model. T5 (Text-to-Text Transfer Transformer) is a versatile encoder-decoder transformer model that frames all NLP problems as a text-to-text generation task. By converting inputs and outputs into textual forms, T5 provides a unified approach to tasks such as translation, summarization, and NLI.

4.3.3. Encoder-only models

Encoder-only models utilize only the encoder part of the transformer architecture. They are mainly designed to understand input text, but they are not designed to generate an output text. They are mainly utilized for text classification or information extraction tasks. BERT (Bidirectional Encoder Representations from Transformers) models belong to this category.

4.3.4. Decoder-only models

Decoder-only transformer architectures are designed to efficiently generate coherent and contextually relevant text. The decoder-only models used in this research are: Gemma model, Llama (Large Language Model Meta AI) model and a Qwen model. Llama models focus on scaling transformer capabilities while optimizing performance on limited computational resources. Qwen models represent a newer family of large-scale language models focused on efficient training and inference, often incorporating optimized attention mechanisms. These models process input tokens in an autoregressive manner, predicting the next token based on the sequence generated so far. Unlike encoder-decoder architectures, decoder-only models do not encode the full input into a separate representation before generation; instead, they incorporate context directly through the self-attention mechanism applied over the growing sequence.

4.4. Training vs Fine-tuning

Training and fine-tuning are important steps in machine learning, both serving different, distinct purposes. Training a model is a more general approach to teach a model, often referred to as pre-training. This learning procedure can be either a supervised learning or an unsupervised learning. A model that was exposed to this kind of training is called a pretrained model. During unsupervised learning, an LLM is exposed to a large text corpus and the goal of the model is to predict the next token in the text: that is, when the model is at token i it doesn't know yet token $i+1$, or anything after that. It makes a prediction of what this next token might be and based on this prediction the cost function is updated. In the next step, the original token $i+1$ is given to the model (not the token predicted by the model) and the prediction step is repeated. However, in practice a procedure called Masked Language Model (MLM) is used. The principle of this idea consists of masking certain words in a given text document. These words are masked in a random manner, appearing as [MASK] or something similar for the language model during pre-training. The portion of the masked words in the text is around 15%, out of the whole corpus. The model is exposed to a large set of texts from different domains, varying from medical, legislative to books and articles, and even to webpages (e.g. Wikipedia is often used to this). The main purpose of the model, during pre-training is to maximize the portion of good predictions, which is achieved by adjusting the model's parameters to minimize a cost function.

This cost function can be of different kinds, the main idea behind it is that it stores how bad the model performs for every example. The cost function often used for transformer models is a cross-entropy loss function between the predicted probability distribution and the true token (in our case, the value of the masked word). In the book "Deep Learning for Natural Language Processing: A Gentle Introduction" by Mihai

Surdeanu and Marco A. Valenzuela-Escárcega [14] the most used cost functions in NLP are formalized. For the purpose of understanding it better, the same notations for the equations will be used in this research. Therefore formalize the cost function, it can be defined as follows:

$$C = -\sum_{i=1}^{m} log\ p(y_i|x_i; \mathbf{W}, \mathbf{b})$$

This cost function is called the cross entropy for a multiclass classification. In next token prediction or masked token prediction a multiclass classification is used, where the possible classes that can be predicted are the words from the entire vocabulary. In the formula y_i is the notation of the correct label of token i. The letter m denotes the number of examples in the dataset. The $\mathbf{W}$ tensor corresponds to the weights to be learned, while the vector $\mathbf{b}$ represents the bias vector. To get the predicted probability a softmax function is used:

$$p(y_i|x_i, \mathbf{W}, \mathbf{b}) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} = \frac{e^{w_i x + b_i}}{\sum_{j=1}^{k} e^{w_j x + b_j}}$$

, where k represents the number of classes to be predicted (in case of token prediction this corresponds to the vocabulary), z_i represents the output prediction, which is calculated using the weight vector w_i, the input embedding x and the bias vector b_i. This formulation measures the negative log-likelihood of the correct token under the model's predicted distribution and is minimized via gradient-based optimization. Other cost functions might be used, however the described cost function belongs to the most popular ones.

In case of transformer-based models, during training the W^Q, W^K, W^V matrices mentioned in the previous section are changed to achieve a better performance.

For the purpose of this paper, here will be described learning algorithms used specifically in transformer-based models, since the experiments are done on transformer-based models. The most used learning algorithm of them is usually some form of the stochastic gradient descent (SGD) algorithm, with some modifications and optimizations (e.g. Adam or AdamW [15]) to fit the transformer architecture. An intuitive analogy of SGD would be to imagine that one stands on a mountain and wishes to arrive in the lowest point of the valley. However, this person is blindfolded so they cannot see where to go. So the only thing this person can rely on is the slope of the ground beneath their feet and take small steps downhill until they reach a local minimum point. In this analogy finding the lowest point of the valley corresponds to minimizing the cost function, and the slope is the gradient of the loss function with respect to the model parameters. The gradient is given by the first partial derivative of the loss function with respect to the current parameter. Taking a step corresponds to updating the model's parameters based on this gradient like: $\theta = \theta - \alpha \frac{d}{d\theta} C(\Theta)$, where θ represents one parameter to be learned (out of all - Θ) and α corresponds to the learning rate, which in terms of the analogy is the size of the step downhill. The word *stochastic* refers to the fact that instead of calculating the gradient using the entire dataset (as it is needed for the standard cost function), one random training example is used or a small sample of examples, which is called a minibatch.

Certain optimizations of this algorithm exist and a learning algorithm in practice usually combines some of those. One of this optimizations is the minibatching. In the traditional stochastic gradient descent, the parameters after each training example is updated, which can result in a fast learning algorithm, however, it has the downside that it

may jump around or even out of the optimal solution. This disadvantage was mitigated with minibatching, which takes a small number of examples and only updates the parameters after the corresponding minibatch via backpropagation (or similar). The learning algorithms most often used in pre-training transformer-based models are variations of an improved SGD algorithm, namely Adaptive Moment Estimation (Adam) [15], it is first formally defined as follows:

```
Require: α: Stepsize
Require: β1, β2 ∈ [0,1): Exponential decay rates for the moment estimates
Require: f(θ): Stochastic objective function with parameters θ
Require: θ0: Initial parameter vector
m0 ← 0 (Initialize 1st moment vector)
v0 ← 0 (Initialize 2nd moment vector)
t ← 0 (Initialize timestep)
while θt not converged do
    t ← t + 1
    gt ← ∇θ ft(θt−1) (Get gradients w.r.t. stochastic objective
    at timestep t)
    mt ← β1 · mt−1 + (1 − β1) · gt
        (Update biased first moment estimate)
    vt ← β2 · vt−1 + (1 − β2) · gt^2
        (Update biased second raw moment estimate)
    m̂t ← mt/(1 − β1^t)
        (Compute bias-corrected first moment estimate)
    v̂t ← vt/(1 − β2^t)
        (Compute bias-corrected second raw moment estimate)
    θt ← θt−1 − α · m̂t/(√v̂t + ϵ)
        (Update parameters)
end while
return θt (Resulting parameters)
```

Two important improvements are integrated in this algorithm compared to stochastic gradient descent, namely the integration of "momentum" and some good features are adapted from the RMSProp approach. The intuition behind momentum is that more recent gradients should influence the direction and the learning more, than older ones (given that the variance of them is small). It achieves this by introducing a new equation to calculate the gradients. So if the previous update rule was like: $\theta = \theta - \alpha \frac{d}{d\theta} C(\Theta)$, where θ represents the model's parameters (both weights and biases), to utilize the notion of momentum, the equation becomes:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

, where g_t represent the gradient, previously noted as $\frac{d}{d\theta} C(\Theta)$. So updating a parameter would be: $\theta = \theta - \alpha \cdot m_t$. The α value represents the stepsize or the learning rate, while m_t would represent the xponential moving average of gradients at this timestamp t. β_1 controls how much of the previous gradients should influence the current momentum, and it is usually set to 0.9.

The Adam algorithm utilizes another optimization, introduced by AdaGrad [16] previously. The disadvantage that this optimization is trying to solve is the problem of how different parameters affect each other. In case when the direction (first derivative) given by one parameter is much steeper than of another parameter, the regular SGD algorithm will more likely move first towards the direction indicated by the steeper parameter, and the other parameter will not affect it as much. This can result in almost like going around the shortest path downhill, which yields to slower learning altogether.

To mitigate this, a new variable and a new equation is introduced:

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

, where β_2 is usually set to 0.99.This is an exponential moving average of the squared gradients. The update rule then would become:

$$\theta = \theta - \alpha \cdot \frac{g_t}{\sqrt{v_t + \epsilon}}$$

, where ϵ is a small value to avoid division by zero. Updating the parameters this way controls the amount by which each parameter will affect the system. It controls it in a way that bigger variances will result in smaller updates (because the variance is in the denominator), while smaller variances will be amplified, it keeps track of the magnitude of gradients for each parameter.

The Adam algorithm uses both equations, the first one is used so that more recent gradients would influence the system more, and the second one to make sure that the different parameter influences are normalized. However, another element must be added, since the first and second moment vectors are initialized with 0, they will be biased towards 0 in the first steps, so an introduction of a bias term is needed. This bias term will be $\hat{m}_t \leftarrow m_t/(1 - \beta_1^t)$ for the momentum term, and $\hat{v}_t \leftarrow v_t/(1 - \beta_2^t)$ for the variance/second term. The update rule then becomes:

$$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

, the nominator being the bias-corrected momentum, giving the direction of the step, and the denominator scales the step inversely to the gradient's variance, preventing overly large updates for parameters with high variance gradients.

While the training procedure described earlier develops a model's general-purpose capabilities, fine-tuning aims to specialize the model for a particular task.

We talk about fine-tuning in a case where one may want to make a model more accurate and precise in one single task. It can be looked at, however as the continuation of the training procedure, when during fine-tuning all parameters of the model are updated to adapt to that specific task. This kind of fine-tuning however is costly and it takes a lot of time, therefore in practice especially for smaller,specific tasks Parameter-Efficient Fine-tuning is used.

4.4.1. Parameter-efficient Fine-tuning

Parameter-Efficient Fine-tuning is used when one does not want to update all of the model's parameters, but only a selected subset of them. The reason for this can be, for example, resource constraints, as updating all of the model's parameters — especially when this number is in the billions — can be very costly in terms of both resources and time. Therefore, it is more efficient to use this method. The parameters, which should not be updated during fine-tuning are frozen. The training algorithm used for fine-tuning is similar to the ones used for pre-training. The loss function used is specific to the task for which the fine-tuning is done.

The Parameter-Efficient Fine-tuning relevant to this thesis in called LoRA, which stands for Low-Rank Adaptation [17].

LoRA freezes the original model weights, then adds an addition: a low-rank matrix. Figure 4.3 illustrates how LoRA modifies a pretrained weight matrix. The base weights $W \in \mathbb{R}^{d \times d}$ remain frozen, ensuring the original pretrained knowledge is preserved. Instead of learning a full $d \times d$ update matrix ΔW, LoRA does the update as

$$\Delta W = BA,$$

with A being of dimension $r \times d$ and B being of dimensions $d \times r$, where $r \ll d$. At initialization, B is set to zero and A is initialized randomly via Gaussian distribution, so that $\Delta W \approx 0$ and the model initially behaves exactly like the base pretrained model. During fine-tuning, only A and B are updated using gradient descent.

Given an input $x \in \mathbb{R}^d$, the pretrained model produces

$$h_0 = Wx,$$

while the LoRA update adds a low-rank correction

$$h = h_0 + \Delta W x = Wx + BAx.$$

This design has two main advantages: First is parameter efficiency, because instead of d^2 trainable parameters in ΔW, LoRA only introduces $2dr$, which is far smaller when $r \ll d$ (r is much smaller than d). The second advantage is that it preserves the pretrained knowledge, because W is frozen, so the model retains its general capabilities, while the low-rank adapters A, B learn task-specific adjustments.

In practice, LoRA makes it possible to adapt very large models to new tasks with only a smaller fraction of the parameters being trained, while still running the model just as fast as before during inference.

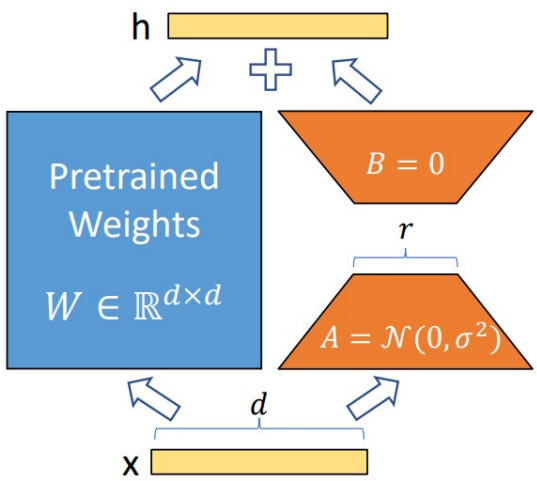

Figure 4.3: LoRA weight matrices [17]

4.5. Dataset manipulation – Wordnet, POS tagging

4.5.1. Part-Of-Speech Tagging

Part-Of-Speech Tagging is the process of assigning the grammatical category of each word in a sentence, like if it is a noun, a verb or an adjective.

4.5.2. WordNet

WordNet([18]) is a large database of English words that is used by both people and computer programs. It was developed at Princeton University in the 1980s by a team led by psychologist and linguist George A. Miller. The goal was to create a resource that did more than a traditional dictionary. It can be thought of as a dictionary, with the difference that, instead of being in an alphabetical order it is organized by conceptual meaning, in so called synsets. The word synset comes from ”synonym set”, which is a collection of words that together represent a certain concept. The words this way share the same meaning in a certain context. For example, “car” and “automobile” belong to the same synset when they refer to a motor vehicle. Words that have more than one meaning appear in more than one synset. This way, WordNet can show the different senses of a word and keep them separate.

It links together synsets, through relations such as synonymy (which means the words have a similar meaning), antonymy (opposite meaning), hypernymy (means generalization), and hyponymy (means specialization). For example, one of the main relationships in WordNet, by which it links together words is the ”is-a” relationships, like a ”dog” is a kind of ”animal”, and a ”german shepherd” is a breed of dogs. This type of relationship is called hyponymy, when we go from a general term to a more specific one, while the inverse direction of it is called hypernymy. Another example is the part-whole relationship, like a ”bike” has a ”wheel”, which is called holonymy.

In the network of WordNet the synsets are the nodes and the relationships between synsets are the link or edges. So words that belong to the same synset are somewhat synonyms, and if we imagine a tree in the WordNet network, then synsets closer to the root, represent generalizations of synsets closer to the leafs on the same branch. The noun, verb, adjective and adverb versions of the same word are represented in different synsets. For example we have the word ”bike”, which has a meaning as a noun and as a verb, so in WordNet they are represented with unique names, like ”bicycle.n.01” for the noun and ”bicycle.v.01” for the verb meaning.

WordNet can be used in many ways. In NLP, it helps with tasks like word-sense disambiguation—figuring out which meaning of a word is intended in a sentence. Other applications might be automatic text classification, text summarization or machine translation.

However, WordNet also has limitations. It mainly covers standard English, so it may not include slang, new words, or highly specialized vocabulary. Sometimes it splits meanings into very fine categories, which can make it harder for people or computer programs to decide which sense fits best.

Also, WordNet lists relationships between words without knowing which ones are most important in a particular piece of text. For work in technical or scientific areas, WordNet often needs to be combined with other, more specialized resources.

4.6. Jaccard Similarity and Distance

Jaccard similarity is a simple way to measure how similar two sentences are. It works by turning each sentence into a set of words and comparing how many words they have in common. The formula is the following:

$$Jaccard\ Similarity = (number\ of\ common\ words)/(total\ number\ of\ unique\ words).$$

The Jaccard Distance is the Inverse of the Jaccard Similarity:

$$Jaccard\ Distance = 1\breve{}Jaccard\ Similarity.$$

4.7. Cosine similarity

Cosine similarity measures the similarity of two non-zero vectors, meaning how similar the direction of the two vectors are. It measures the cosine of the angle between the two vectors, given by the equation:

$$cos(\theta) = \frac{A \cdot B}{||A|| \cdot ||B||} = \frac{A \cdot B}{\sqrt{\sum a_i^2} \cdot \sqrt{\sum b_i^2}}$$

, where θ is the angle between vector A and B. In Natural Language Processing it is used to determine the similarity of two texts. It is measured as the cosine similarity of two embedding representations, and it can vary from -1 to 1:

- 1 meaning that the two vectors point in the same direction,
- 0 meaning that they are orthogonal to each other, having no directional similarity,
- -1 meaning they point to exactly opposite directions.

In NLP, if the cosine similarity of two vector embeddings are closer to 1 it indicates that the meanings are closer, while if the cosine similarity is closer to -1 it indicates antonymy.

Chapter 5. Detailed Design and Implementation

5.1. General approach

This chapter describes the detailed design and implementation steps of my project. The objective of this work was to analyze and improve the NLI performance by augmenting the SICK dataset and testing multiple LLMs on the task. My approach consists of the following parts:

1. Performing an analysis on the SICK dataset to identify potential biases or artifacts that could affect model training or evaluation.
2. Augmenting the dataset by replacing subjects in the premises with hypernyms and synonyms using POS tagging and WordNet, aiming to increase lexical diversity.
3. Fine-tuning the Gemma 3 Instruct model on NLI data to assess its performance and improve its ability to handle variations.
4. Evaluating different smaller LLMs locally using LM Studio, to compare performance and behavior on NLI.

5.2. Analysis on the dataset

The main reason for the dataset analysis is the identification of potential biases. It is an important part of my research because if the dataset itself is biased, the results might be biased too. The approach for this contains some parts.

Firstly, as in the paper [4] the authors note that there exists a bias in the SNLI dataset, namely some annotated artifacts. For example, certain words often appear in specific labels — contradiction sentences often include words like "never" or "no," neutral ones may have more modifiers or extra details, and entailment sentences are usually shorter and look very similar to the premise. This leads to a hypothesis-only bias in the dataset, and because of this some text-classifiers were able to correctly classify more than half of the examples, only based on the hypothesis. While the SICK dataset was made specifically with the purpose of eliminating the existing biases in SNLI, it is important to analyze this dataset too, to make sure that it is free from artifacts, same or different as in the SNLI dataset.

To lead this analysis the first step was to compare the sentence lengths across the labels, to make sure that the models cannot distinguish between labels based solely on the length of the sequence (e.g. "gurarangan" showed that in the SNLI dataset the length of a neutral hypothesis tends to be longer, while the hypothesis of an entailment pair was significantly shorter). The implementation of this step was done in Python language.

To investigate whether the sentence lengths vary across the entailment labels, I analyzed the character lengths for the premise (column named as "sentence_A") and the hypothesis (column named as "sentence_B") separately. The dataset was loaded from the SICK_train.txt file using pandas, and sentences were grouped based on their associated

entailment labels: NEUTRAL, ENTAILMENT, and CONTRADICTION. Three subsets of the data were created by filtering the dataset based on the "entailment_judgment" column. This allowed for a comparative analysis of sentence lengths across different entailment types. We plotted histograms of sentence lengths (measured in number of characters) for both the premise ("sentence_A") and the hypothesis ("sentence_B"). Separate histograms were overlaid for each entailment type using color-coded bars to visualize their distributions: premise is displayed in the top panel, while the hypothesis is displayed in the bottom panel. All visualizations were generated using matplotlib, and the character length of each sentence was computed using the *str.len()* method on the respective sentence columns.

While the previous step could be a good initial indicator, a better approach is to compare the word count instead of the character count. This step was essentially the same as the previous one, the only difference being that instead of using the *str.len()* method on the different sentence columns, I used the *str.split().map(lambda x: len(x))* method which gives the word count of each sentence. The results were displayed the same way.

Another analysis made is comparing the difference in word count between the premise and hypothesis across each label. To do this, a function is used, which simply calculates the word count of the premise and hypothesis separately, then appends the difference to a list. This function has one single parameter which is the dataset itself. Before calling the function the SICK dataset is split in three parts based on the label, as mentioned previously, then the function is called for each subset of the data. The frequency of word count differences is displayed in a histogram in the same way as for the previous two steps. The range for the histogram spans from the minimum of all word count differences across the three entailment labels to the maximum of all those differences.

As another approach, besides the length of the sentences, I wanted to take a look at the similarities between the premise and hypothesis in a sense of phrasing and word similarity. The motivation behind this analysis is that a significantly higher word overlap between the premise and hypothesis in one particular label could indicate the presence of an artifact in the dataset. A model that learns to rely on such overlap might perform well without actually engaging in logical reasoning. Therefore, avoiding these types of artifacts helps ensure a more accurate estimate of a model's true inference capability. To achieve this, I compared the Jaccard similarity for each premise-hypothesis pair across the labels. To implement this, I first calculated the Jaccard similarity in a function, based on the set intersection divided by the set union of two sets. Then, in a separate function, I split a given premise and hypothesis into sets of words. In a third function, I connected the two previous methods by traversing the given dataset line by line, constructing the sets of words for each sentence pair, and calculating their Jaccard similarity. This function is then called for each subset of the dataset, and the results are displayed in a histogram in a similar way as described previously.

5.3. SICK dataset – augmentation

The motivation behind this dataset modification was simple: to see whether the models can understand the semantic meaning of a word. For example, if the subject of the premise is a different word than the subject of the hypothesis, it is an indicator that the two sentences are about different topics, which might indicate that the relationship

between them is neutral. In the same way, if the subject of the premise and the hypothesis is the same word, then it is an indicator that at least the context might be similar in the two sentences. However, if the subjects of the two sentences have the same meaning, but different words are used to describe the same notion, then the model has to have an "understanding" of each word, and has to know that they describe the same thing to provide an accurate response/prediction.

To investigate this, I aimed to replace the subject of each premise/hypothesis in the SICK dataset with its synonym/hypernym.

For the first step, which is identifying the grammatical subject of each premise/hypothesis in the dataset, I used a POS (Part-of-Speech) tagging approach (shown in segment 5.1). It relies on the spaCy NLP library to parse sentences and determine their syntactic structure. The program begins by loading the small English spaCy model, which can tokenize sentences, assign part-of-speech tags, and it can detect dependency relations between words. For each sentence, it searches for a token whose dependency label is "nsubj", which indicates that the word functions as the nominal subject of the sentence. If such a token is found, the word is returned; if not, the function returns the value None.

```
import spacy

def getSubjectOfSentence(sentence):
    nlp = spacy.load("en_core_web_sm")
    doc = nlp(sentence)
    for token in doc:
        if token.dep_ == "nsubj":
            return token.text
    return None
```

Listing 5.1: POS tagging

To handle multiple sentences, the script processes them one by one, collecting all detected subjects into a Python set to ensure that each subject is stored only once, regardless of case differences. Before subject extraction, the dataset is read using the pandas library. The column names are stripped of any leading or trailing spaces to avoid lookup errors. The script then iterates over each row, retrieving the "premise" sentence from the sentence_A column, the "hypothesis" sentence from the sentence_B column, and the corresponding entailment label from the entailment_judgment column. The subject is extracted only from the premise sentence, converted to lowercase, and added to the set of collected subjects. Then the index of each processed row is printed to track progress.

Once all rows have been processed, the set of unique subjects is returned. In essence, this code reads a dataset of paired sentences, uses spaCy's dependency parsing to locate the grammatical subjects of the sentences, and outputs a case-insensitive list of distinct subjects found across the entire dataset.

In the second part of the project, I used WordNet to find hypernyms (more general terms) for each subject. The script begins by importing the required libraries. It uses NLTK (Natural Language Toolkit) to access WordNet and perform tokenization and part-of-speech tagging, inflect to handle plural and singular word forms. A predefined set of subject words is stored in a Python set, representing the distinct entities extracted previously. The function *findClosestHypernym (word)* searches WordNet for noun senses (synsets) of the given word. For each synset, it retrieves hypernyms along with their "distance" — a measure of how far the hypernym is from the original word in the semantic hierarchy. A distance of 1 means the hypernym is immediately above the word in the

hierarchy, making it the closest general category. When such a hypernym is found, the function returns its first lemma (canonical word form), replacing underscores with spaces for readability. If no suitable hypernym is found, the function returns the value None.

The second function, *makeHypernymMap(subjects)* (5.2), processes all the subjects in the input set and builds a mapping between each subject and its closest hypernym. Since many words in the subject set are plural, the function first checks if the word is plural by using the inflect library to obtain its singular form. The singular version is used for the WordNet search, because WordNet entries are stored in singular form. If a hypernym is found for the singular form, it is converted back to plural if the original subject was plural. The result is stored in a dictionary where each original subject word is linked to its generalized category. The same process is repeated similarly to find hyponyms and synonyms of the subjects. The hyponyms being a more specific term of the initial word. In the hyponym version, the search in WordNet is reversed: instead of retrieving the closest hypernym (a broader category), the code takes the first hyponym from the first synset of the word, resulting in a more specific term.

```
import nltk
from nltk.corpus import wordnet
from nltk.tokenize import word_tokenize
from nltk import pos_tag
import pandas as pd
import spacy
nlp = spacy.load('en_core_web_md')

import inflect

p = inflect.engine()

def findClosestHypernym(word):
    for syn in wordnet.synsets(word, pos=wordnet.NOUN):
        for hypernym, dist in syn.hypernym_distances():
            if dist == 1:
                return hypernym.lemmas()[0].name().replace("_", " ")
    return None

def makeHypernymMap(subjects):
    hypernymMap = {}
    for subject in subjects:
        #checkking if original word is plural
        singular_form = p.singular_noun(subject)
        is_plural = bool(singular_form)
        base_word = singular_form if is_plural else subject

        #gett hypernym for singular form
        hypernym = findClosestHypernym(base_word)

        if hypernym:
            hypernym = p.plural(hypernym) if is_plural else hypernym

        hypernymMap[subject] = hypernym
    return hypernymMap
```

Listing 5.2: Hypernym selection with WordNet

In the synonym version, WordNet is again queried for lemmas of the given word, but with additional filtering to remove exact matches, plural/singular variants, proper

nouns, and multi-word expressions. The remaining candidates are then compared to the original word using spaCy's word vectors, and the synonym with the highest semantic similarity score is chosen.

To make sure the grammar remained correct, I checked if the original subject was plural and, if so, handled the singular form during the lookup. Once I found a suitable hypernym, I converted it back to plural if needed and stored the result in a mapping (dictionary) that linked each original subject to its hypernym.

Finally, another Python script was created (*replace_subject_synonym.py*) to actually replace the subjects with the corresponding hypernym, hyponym or synonym. This script begins by loading the dataset and e.g. the hypernym mappings, then it iterates over the rows of the dataset and identifies the subject of the sentence in the same way as used in the POS tagging part mentioned previously. When it finds the subject it replaces it with the corresponding hypernym from the loaded mapping. If there is no corresponding hypernym for the subject it puts back the original subject of the sentence. The modified sentence and the other columns are appended to a Python dictionary. In the end the modified dataset is saved to a txt file.

5.3.1. Improved synonym substitution

While the solution so far gets good results for finding good hypernyms for the sentences, to find a suitable synonym for a certain word takes more than that. For a suitable syonym it is not enough for us to take into account only the word meaning and the closest synsets in the WordNet corpora, it is much more important to take into account the context in which that certain word, that we wish to replace, is used. For example, the same word can have different meanings depending on the context, like: the word "bat" means both the nocturnal animal and the object made of wood that is used to hit a ball, the word "nail" refers to both the part of one's finger and to the piece of metal. In these cases if we use the previously defined hypernym algorithm as is, for synonym substitution, the words with the highest similarity score that the WordNet will suggest for the word "bat" are: "cricket bat" (with score 0.7104), "at-bat" (with score 0.4855), "squash racquet" (with score 0.1692). As one can predict, if the context in which the word "bat" is used refers to the animal meaning of the word, by simply applying WordNet synonyms will not get the desired result.

To solve this problem, a new solution was proposed which takes into account the contextualized meaning of the word, and uses cosine similarity to find the best matches of the synonyms. The diagram 5.1 shows the pipeline of the approach:

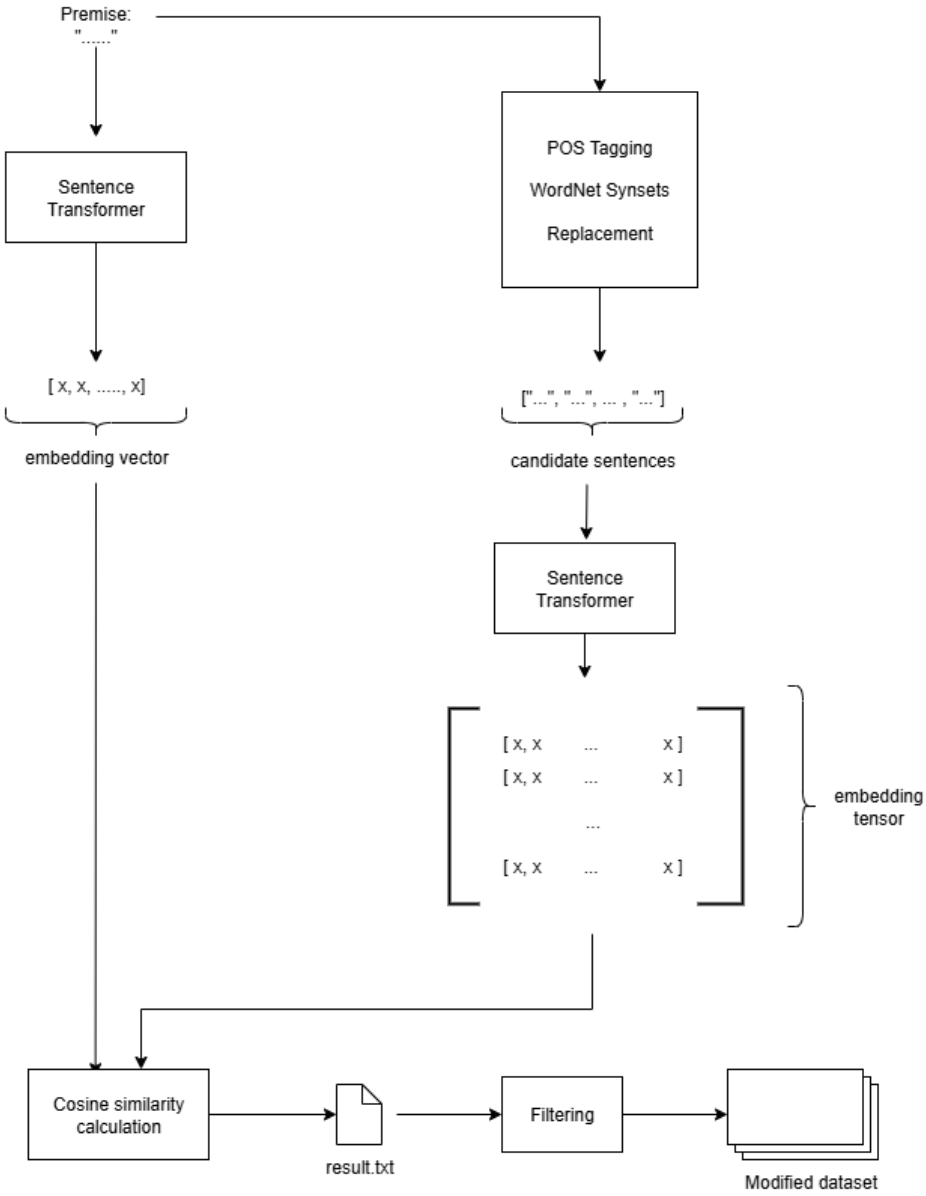

Figure 5.1: Pipeline of synonym substitution solution

First, I used a Sentence Transformer, namely "sentence-transformers/all-MiniLM-L6-v2" to compute a contextualized embedding of an entire sentence, which represents the "essence" of the sentence. Then, I made use of the cosine similarity to compare the candidate sentence embeddings to the original sentence embedding.

The code used in the local Python environment begins by defining the sentence transformer model that will be used and defining an inflect engine, which is used for plurality checks. Two functions follow, which are used to find the subject of the sentence and to check the plurality of a word (the *find_subject_of_sentence()* and *is_just_plural_form()* functions). These functions are the same POS tagging and plurality check functions used in the original hypernym substitution solution.

The function named *embedding_nearest_neighbors()* (shown in segment 5.3) contains the main logic of this solution. The function gets as parameters the sentence that we want to replace, the subject of that sentence (variable called *word*) and a parameter

called *top_k*, which represents the number of the candidates we want to take into account further on. The function first computes the contextualized embedding of the given original sentence, using the sentence transformer's *encode* method. Then, it traverses the synsets associated to the given word, and constructs a candidate list of words. It is important to note here, that every word that has a synset association to the given word is taken into account. The next step this time is to replace the subject of the original sentence one-by-one with the candidate words, constructing a candidate sentence list. During this substitution, I took each modified sentence and calculated the embedding of that sentence with the same *encode* method of the sentence transformer. Using the original sentence embedding and the new sentence embedding, the next step was to calculate a cosine similarity of the sentences, which was computed for each candidate sentence, creating a list of candidate sentences, where each candidate is associated with a cosine similarity score. The candidate list was sorted based on the cosine similarity score, using Python's built-in *sorted()* method. During this process the candidate word that is identical to the original word is discarded.

```
def embedding_nearest_neighbors(sentence, word, top_k=10):

   #get the embedding of the sentence
    query_emb = model.encode(word, convert_to_tensor=True)
    sent_emb = model.encode(sentence, convert_to_tensor=True)


    candidates = set()
    for syn in wn.synsets(word, pos=wn.NOUN):
        for lemma in syn.lemmas():
            candidates.add(lemma.name().replace("_", " "))

       #additional exploration of hypernyms and hyponyms too, to better cover
        for hyper in syn.hypernyms() + syn.hyponyms():
            for lemma in hyper.lemmas():
                candidates.add(lemma.name().replace("_", " "))

    candidates = list(candidates)
    results = []
    for candidate in candidates:
        if candidate.lower() == word.lower() or word in candidate:
            continue
        cand_sent = sentence.replace(word, candidate)
        cand_emb = model.encode(cand_sent, convert_to_tensor=True)

        similarity = cosine_similarity(sent_emb.reshape(1, -1), cand_emb.reshape(1, -1))[0][0]

        results.append((cand_sent, similarity))

   #sort by descending similarity
    results = sorted(results, key=lambda x: x[1], reverse=True)[:top_k]
    return results
```

Listing 5.3: Contextualized synonym search function

The function called *find_best_in_context()* connects the two previous functions:the POS tagging with the contextualized embedding-based synonym search. The improved solution was tested for 100 examples in the local Python environment.

Due to computational resource constraints, the local Python environment is insufficient for using the previous solution to modify the entire SICK dataset. Therefore, the decision was made to run the implementation in a Kaggle notebook. Kaggle was chosen, because it provides a 30 hour GPU usage per week, for any user.

To make use of the GPU provided by Kaggle, a modification was made to the solution (5.4) : instead of calculating the sentence embedding of each word separately, first, the whole list of candidate sentences is created, then that list is encoded into embeddings with one single call in batches of size 32 (used for processing contradiction examples) and 64 (used for processing neutral and entailment examples). This way, a tensor is created with the embedding vectors. Each row in the final tensor corresponds to one sentence embedding, and each column corresponds to one dimension of the embedding vector. Instead of calculating the cosine similarities one-by-one, PyTorch's *F.cosine_similariy* method is used to compute the similarities in a single, vectorized operation. The resulting candidates with the scores are sorted in the same way.

```
def embedding_nearest_neighbors(sentence, word, top_k=10):
    ...

    cand_embs = model.encode(cand_sentences, convert_to_tensor=True,
   batch_size=64)

    similarities = F.cosine_similarity(sent_emb.unsqueeze(0), cand_embs
   )

    results = list(zip(cand_sentences, similarities.tolist()))

    results = sorted(results, key=lambda x: x[1], reverse=True)[:top_k]
```

Listing 5.4: Change in Kaggle

The top 3 candidates are selected from the list and saved in a .txt file together with the original sentences.

The next step in the pipeline is to additionally filter the candidates from unsuitable words. For example, in the previous solution it was specified that the synonym cannot be the same word as the original subject, but in from WordNet, there were still result like the word "KID" for the original subject "kids", so additional filtering was needed. This process was done locally, in the Python environment.

The script named *select_best_candidate.py* was responsible for this purpose. The function *select_best_candidate()* gets as parameter the path for the .txt file with the top 3 candidates from the previous step and processes it, choosing the most accurate synonym by calling the function *is_suitable()* for each candidate in descending order, based on their score.

The *is_suitable()* function (5.5) is responsible for filtering the candidates, first by checking whether the subject of the original sentence and the subject of the candidate sentence are actually the same word, but in lowercase or uppercase or in a different plurality. I used WordNetLemmatizer for this purpose. Then in the "constuction" phase of this function, if the word is considered suitable, the plurality of the word was checked, to match the original subject. If the original word was a singular word, the candidate word was transformed to a singular form, and vice versa. The correct form was substituted in the sentence.

```
def is_suitable(original, candidate):
    subject_original = find_subject_of_sentence(original)
```

```
    subject_candidate = find_subject_of_sentence(candidate)
    #if one of the subjects doesn't exist, return none
    if not subject_original or not subject_candidate:
        return False, None

    #lowercase and lemmatization
    subject_original_lemma = lemmatizer.lemmatize(subject_original.
   lower())
    subject_candidate_lemma = lemmatizer.lemmatize(subject_candidate.
   lower())

    #if they reduce to the same lemma, it is not suitable
    if subject_original_lemma == subject_candidate_lemma:
        return False, None

    if subject_original == "group" and subject_candidate == "people":
        return False, None

    orig_as_sing = p.singular_noun(subject_original)
    cand_as_sing = p.singular_noun(subject_candidate)

    if orig_as_sing and cand_as_sing:
        return True, candidate
    elif orig_as_sing:
        cand_plural = p.plural_noun(subject_candidate)
        new_candidate = candidate.replace(subject_candidate,
   cand_plural, 1)
        return True, new_candidate
    elif cand_as_sing:
        new_candidate = candidate.replace(subject_candidate,
   cand_as_sing, 1)
        return True, new_candidate
    else:
        return True, candidate
```

Listing 5.5: is_suitable() function

An additional filtering was done, to exclude the candidate word "people" for the original word "group", which was hard-coded. The reason for this is that, the closest synonymy relationship for the word "group" in the WordNet corpora if oftentimes the word "people", but in our case when the subject is "a group of friends", then the word "group" shouldn't be substituted with the word "people".

A new dataset was created, by taking each premise of the SICK dataset, finding it's best candidate based on the process described above, and using that as the new premise for the dataset. The hypotheses were not modified.

5.4. Models from LM Studio

Throughout the research, for testing models capabilities I used the models locally, via LM Studio, which is a desktop application that allows people to run and test LLMs locally on their computer. LM Studio was used in this project to evaluate smaller language models on the NLI task. These models were selected not only due to limited computational resources, but also because it is increasingly important that smaller models perform well. Ensuring their effectiveness can expand the accessibility and usability of language technologies, especially in resource-constrained environments. Most of the

models used in this study are generative by design, intended primarily for open-ended text generation rather than classification tasks like NLI. However, strong logical reasoning remains a crucial capability even for generative models, especially in tasks that require understanding and inference. LM Studio provided a practical and hands-on environment for testing these models, making it a suitable and efficient solution for the goals of this research.

5.4.1. Models

The models used from LM Studio are the fllowing: Gemma 3 it with 1b parameters, Llama 3.2 with 1b parameters, Qwen 3 with 600 million parameters and a T5 model with 248 million parameters.

5.4.2. API used for testing

LM Studio offers an API, which makes it possible to communicate with the model using Python code. This is a prompt-like communication, where one has to define messages to be sent to the model and there is an URL given in LM Studio, which then can be used to create a prompt to send to the previously loaded model in LM Studio. For example:

```
curl http://localhost:1234/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gemma-3-1b-it",
    "messages": [
      { "role": "system", "content": "Always answer in rhymes.
                Today is Thursday" },
      { "role": "user", "content": "What day is it today?" }
    ],
    "temperature": 0.7,
    "max_tokens": -1,
    "stream": false
}'
```

To evaluate the performance of language models on the NLI task, a custom scripts were developed to interface with LM Studio's local API and get an accuracy for this task (besides the actual post request to the API for which I used as a guideline the tutorial from "reference the site"). The process involves sending sentence pairs to a model and interpreting its responses as NLI labels. As the URL indicates, a function was created for making an HTTP POST request to the model's API. This function constructs a payload containing the model name, the list of messages to be sent and additional information as temperature, maximum number of tokens and the stream. The temperature controls the randomness of the model's output — a lower value like 0.3 makes the responses more deterministic and focused, which is desirable for classification tasks like NLI. The max_tokens parameter defines the maximum number of tokens the model is allowed to generate in response; in this case, it is kept low because the expected response is just a single word. The stream option is set to false to return the full response at once, rather than as a stream of tokens — this simplifies post-processing and is more appropriate when a short, complete output is expected.

The message itself is structured using the roles defined in the API: a "system" role, followed by a prompt specifying what is expected of the model, and a "user" role followed

by the actual message - the sentence pair passed to the function. The prompting strategy was designed to enforce a concise, one-word output. Variations of this prompt and their effects on model performance are discussed in the following chapter.

Then with the

```
response = requests.post(lm\_Studio\_url, headers=headers,
data=json.dumps(payload))
```

line a post request is sent towards the model, which is loaded in LM Studio.

The *get_nli_result()* function sends the request using aiohttp, a library that supports asynchronous HTTP communication. The function returns the predicted label, which is then evaluated for correctness in the *compute_accuracy()* function. This function takes as input the path to a dataset (in CSV or txt format), traverses each row, and sends the premise-hypothesis pair to the model. The prediction is cleaned of any extraneous formatting (e.g. whitespace, punctuation) and compared to the ground-truth entailment_judgment in the dataset. A running total of correct predictions is kept in order to calculate the final accuracy.

Asynchronous execution (async/await) was used to improve efficiency, especially when dealing with a large number of sentence pairs. By using aiohttp along with an asynchronous task scheduler and a semaphore for rate-limiting, multiple requests can be sent concurrently rather than sequentially. This reduces overall runtime significantly, as the program does not need to wait for one request to finish before starting the next. This approach makes the evaluation process scalable and responsive, even when running multiple evaluations on a local machine.

To provide a more detailed analysis of model performance, a confusion matrix is also generated using pandas and seaborn libraries, helping visualize which types of judgments the model most frequently misclassifies. This process was repeated for every model used in this experiment, with minor modifications (e.g. in the prompt to get a valid response from the respective model)

5.5. Fine-tuning with LoRa

To get a better insight on the Gemma model's performance on the NLI task, I performed parameter-efficient fine-tuning (PEFT) rather than full-model fine-tuning. Full fine-tuning updates all model parameters, which for a model of Gemma's scale (1B parameters) is computationally expensive and memory-intensive. In contrast, PEFT methods such as LoRA (Low-Rank Adaptation) introduce a small set of trainable adapter parameters into the network while keeping the original weights frozen. LoRA works by decomposing the update matrices into two low-rank matrices A and B, significantly reducing the number of trainable parameters and the associated memory footprint.

For this experiment, I used the google/gemma-3-1b model from Hugging Face's Transformers library. Since I wanted to fine-tune it for NLI, the first step was to load the dataset and then format it in a way this specific model requires it for fine-tuning.

I begin this process by loading the dataset and filtering out the examples with missing labels. After this the conversion was next into a format this instruction-tuned Gemma model needed. It requires a formatted "text" field, which contains $< bos >$ for the start and $< eos >$ to signal the end of the sequence. The text is formatted that it first contains the description of the text, then the two sentences and in the end, the label. The text is enclosed between $< bos >$ and $< eos >$.

The fine-tuning dataset was a sampled version of the SNLI corpus. The sampling was done randomly and the sample size of the train dataset of SNLI was:

10,000 examples for the training set,

1,000 examples for the validation set, and

1,000 examples for the test set.

After preparing the dataset, the model was loaded from HuggingFace in 4-bit precision via the BitsAndBytes library, a form of quantization that stores weights using fewer bits per parameter. Instead of loading weights in full precision (16/32-bit), the model weights are quantized into 4-bit integers to reduce GPU memory consumption and speeds up training while preserving most of the model's representational capacity. The quantization was applied only to the base model weights — the LoRA adapter parameters remained in higher precision. After loading the model, the sampled dataset was tokenized using the Gemma tokenizer.

During fine-tuning, only the LoRA adapter weights were unfrozen and updated; the rest of the model parameters remained fixed, meaning the backpropagation step and optimizer updates only touched a very small subset of the parameters. This selective training greatly reduced computation and allowed the fine-tuning to fit within Google Colab's hardware limits. The parameters of LoRA were set according to the sequence 5.6:

```
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1,
    bias="none",
    task_type=TaskType.SEQ_CLS
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
```

Listing 5.6: Configuration of LoRA

The parameter r sets the size of the LoRA matrices, the parameter lora_alpha is responsible for scaling the LoRA module before adding it back together with the original parameters of the model. The lora_dropout is used for regularization, which means that during training some portion (indicated by the lora_dropout parameter) of the neurons are randomly "turned off", preventing the model to rely greatly on specific neurons. By specifying the bias as "none", we say that in the fine-tuning process the bias terms of the model shouldn't be updated. Since the task of NLI is a sentence classification task, we specify the task_type as "SEQ_CLS" [19].

The optimizer used was paged AdamW with 8-bit optimizer states, provided by the bitsandbytes implementation. AdamW is an extension of the Adam optimizer that decouples weight decay from the gradient update, improving generalization and stability. Training was performed for 3 epochs with a batch size of 4 and a learning rate of 510^{-5} (set as default) .

Training was run entirely in Google Colab, which provided a suitable balance of available GPU memory and ease of setup.

5.6. Converting Fine-tuned model to GGUF format

To be able to use the fine-tuned model locally as the other models from LM Studio, it was necessary to convert it into a .gguf format. To do this a predefined converter by llama was used.

Before the conversion, a custom Python script was used to merge the fine-tuned LoRA weights with the base model weights. The merging itself was done with the *merge_and_unload()* method provided by the PEFT library from Hugging Face. This method integrates the adapter's fine-tuned weights into the base model parameters, producing a standalone model. As a result, the adapter is no longer needed at inference time.

Chapter 6. Testing and Validation

This chapter contains the results of the experiments.

6.1. Dataset analysis results

6.1.1. Sentence length analysis

The histogram 6.1 represents the sentence length distributions across the labels, more specifically the word count of the sentences.

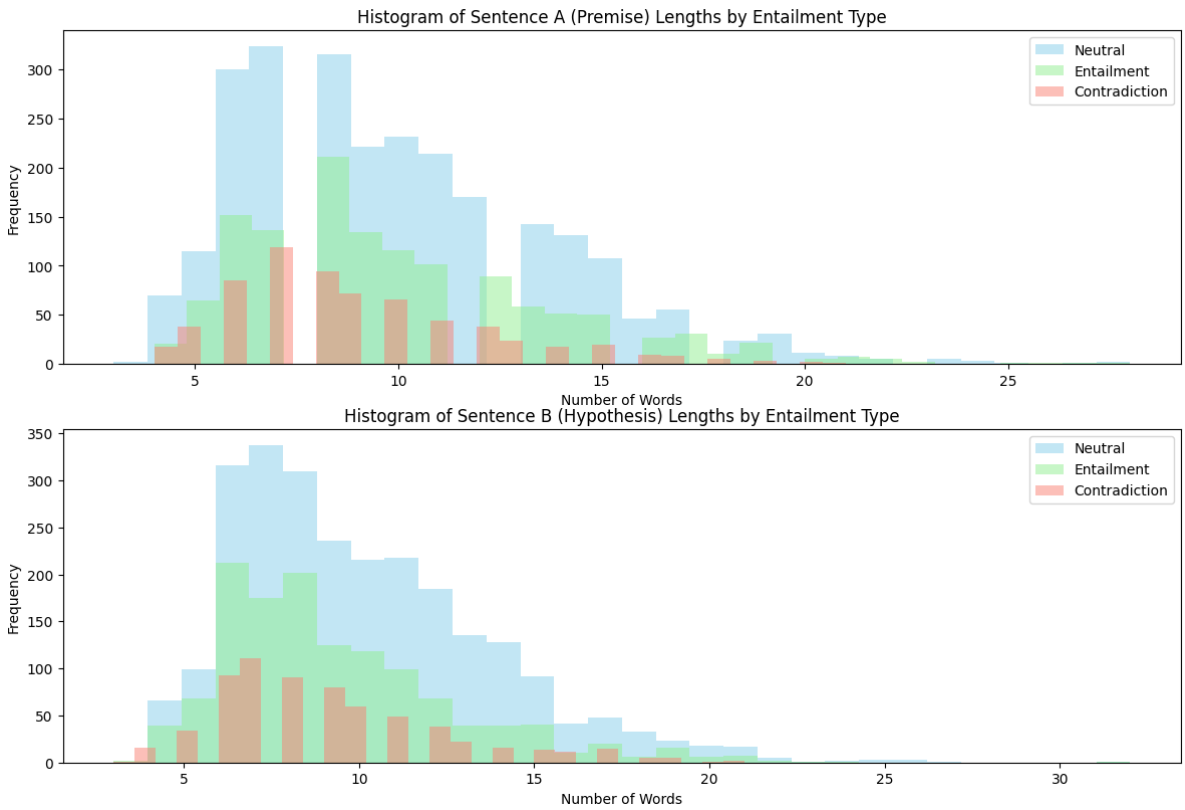

Figure 6.1: Sentence lengths in SICK dataset - in words

The hypothesis for bias was that if for one label (e.g. contradiction examples) the frequency of shorter sentences are much higher than for longer sentences, while for the other labels the exact opposite case is true, that could indicate an artifact in the dataset. As the histograms show, the shapes are quite similar for all labels, which suggests that this kind of artifact is not present here.

The different volume of the frequencies are due to the fact that in the SICK dataset there are twice as many entailment examples as contradiction examples; and around twice

as many examples with the label neutral, than with the label entailment. More specifically there are 2536 neutral, 1299 entailment and 665 contradiction examples in the dataset SICK.

The histogram 6.2 displays the distribution of the length difference of the premise-hypothesis pairs across the labels.

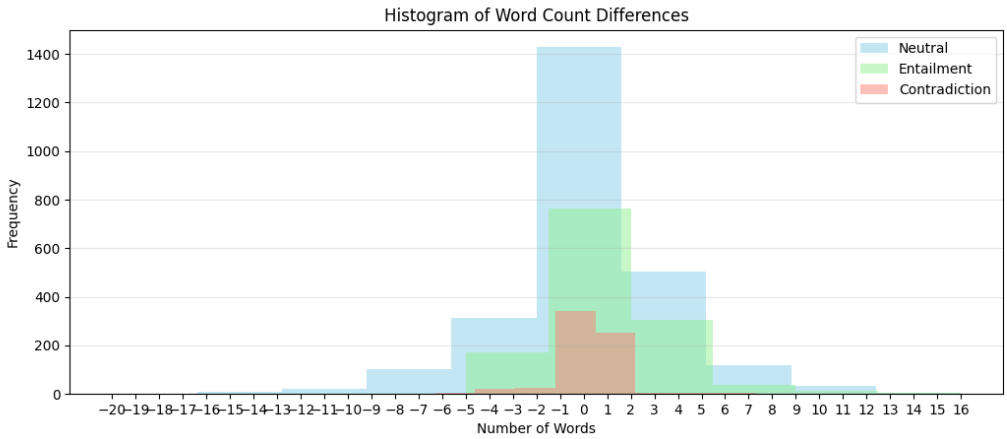

Figure 6.2: Difference between the length of premise and the length of hypothesis for each pair

The difference in length between the hypothesis and the premise can cause another artifact. When the hypothesis is often significantly smaller than the premise in one particular label, it provides a shortcut for the model to predict the correct label without much reasoning. Since the shapes on this histogram are similar, aside from the volume difference, it indicates that the lengths of the sentences are mostly balanced: the highest frequency being a difference of 1 or 2 words. The difference is calculated as length of premise minus the length of hypothesis, so a value of -1 on the histogram means that the hypothesis is one word longer than the premise.

6.1.2. Jaccard Similarity

The histogram 6.3 illustrates the distribution of Jaccard similarity scores between the premise and hypothesis for each example in the SICK dataset, grouped by their entailment labels: ENTAILMENT, NEUTRAL, and CONTRADICTION.

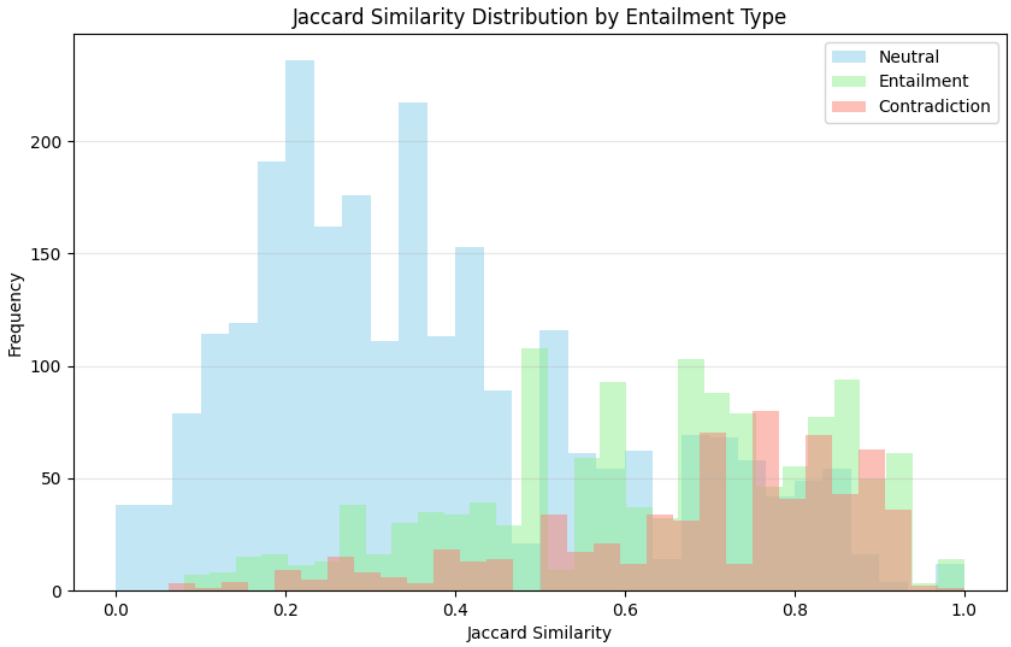

Figure 6.3: Jaccard Similarity - SICK dataset

We observe that neutral examples tend to have lower Jaccard similarity scores, usually around 0.2 to 0.4. This suggests that, in many neutral cases, the premise and hypothesis sentences have limited lexical overlaps, likely due to their weaker semantic connection or topical differences.

In contrast, entailment and contradiction examples show a broader distribution with similarity scores often ranging between 0.6 and 0.9, indicating a higher degree of word overlap between premise and hypothesis. This highlights the fact that both entailment and contradiction typically involve sentences that refer to similar things or events, with differences coming from more subtle changes in meaning, for example, negation or role reversal.

The distribution suggests that models might learn to identify neutral examples simply by comparing the word overlap between the premise and hypothesis. This raises the question of whether this is desirable, as it could lead to a dataset artifact rather than a genuine understanding of sentence meaning. If a model learns to rely too much on surface-level cues, it may not generalize well to more complex or balanced datasets.

6.2. Gemma vs gemma fine-tuned results

Table 6.1 contains the accuracy of the Gemma 3 base model from LM Studio compared to the accuracy of the fine-tuned Gemma model.

There is an overall increase in accuracy of the fine-tuned model, but there is a significant accuracy drop on the contradiction examples of the fine-tuned model compared to the base model. This could indicate a potential bias towards the neutral label, since that label is the one where the accuracy is the highest for those examples.

The prompt used for these tests is the following:

```
Classify the relationship between a premise and hypothesis.
```

```
    Respond with ONLY ONE WORD:\n
    - 'entailment' if the hypothesis logically follows from the premise
   \n
    - 'contradiction' if it contradicts the premise\n
    - 'neutral' if there's no clear relationship
```

Table 6.1: NLI accuracies

Test	Gemma 3 (1b)	Gemma 3 fine-tuned (1b)
Accuracy on SICK	0.41	0.59
Accuracy on Neutral SICK	0.41	0.75
Accuracy on Entailment SICK	0.28	0.46
Accuracy on Contradiction SICK	0.70	0.25

6.3. Dataset augmentation results

The dataset augmentation was conducted to get a better insight into the model's understanding of concepts, to make sure that it does not rely on artifacts, such as word overlap when reasoning about the relationship between a premise and hypothesis pair. Two main experiments were conducted: substituting with hypernyms and substituting words with synonyms.

The hypernym substitution was adequate to test the model on the neutral examples, but substituting the entailment premises with hypernyms could break the logical relationship between the two sentences. Since the hypernym refers to a more general term of the original word, substituting the premise's subject with its hypernym in an entailment example would mean that the first sentence refers to a general concept, while the hypothesis refers to a more specific concept of the same term, indicating a deduction from general to specific, which contradicts the rules for logical entailment. Therefore, in the results table 6.2, only the results for the neutral examples are included to avoid confusion.

The same solution that is used for the hypernym substitution was not enough to modify the dataset with synonyms of the subjects, because the word-level based embeddings to find synonym oftentimes yielded in undesired and improper results, like: "musician", when the original word is "player", referring to a basketball player for example, or the term "Church Father" was suggested for the word "father". A potential bias in the WordNet synsets was obeserved, because oftentimes the term "human being" was suggested for the word "man", but the term "fair sex" for the word "woman" was identified. This can be due to the fact that WordNet might aplify already existing biases in the corpora that it was trained on.

In the improved dataset substitution version not all the premises were modified. There were cases when the POS tagging algorithm couldn't find a subject for the sentence, in this case the premises remained unmodified. The reason for not finding a synonym oftentimes included examples where the subject is a group of words, or the sentence is in a passive form (e.g. "group of kids"). There were also cases where the WordNet search algorithm couldn't find a synonym for the word. This was only the case for a very few subset of the examples. Therefore, the data proportion of the new substituted dataset is:

out of 665 contradiction examples, 175 such examples were, where the algorithm did not find either the subject or a suitable synonym. Out of 1299 entailment examples, there were 106 such cases, while for the 2536 neutral examples, there were 212 such examples. In proportion, around 73% of the contradiction, $\tilde{9}$0% of both the entailment and neutral examples got modified. In total, around 89% of the entire dataset were modified.

One interesting result of WordNet that is worth mentioning, is the case when the original subject was identified as the word ”group”, as in ”a group of kids”. In almost every case, when WordNet was faced with the word ”group”, the closest synonyms that it could identify was the word ”people”, even in the solution, when the algorithm took into account the contextualized embedding of the entire sentence. This effect is likely due to the fact that the semantical meaning of the word ”people” in WordNet is understood as reffering to a mass, or nation. This effect in the filtering process was eliminated with hard-coded filtering.

6.4. Gemma results for augmented dataset

Table 6.2 shows how accuracy changes when we modify the dataset in one way or another. Firstly, we have an overall accuracy, then the second row contains the accuracy for the neutral examples.

In the third row, the subject of each premise was replaced with its hypernym. We observe a small drop in accuracy, though it is not significant.

Finally, the fourth row displays the results for when only the hypothesis was given to the model, and the premise was replaced with an empty string. Interestingly, for the base Gemma-3-1b-it model, the accuracy in this hypothesis-only setting is slightly higher than when both the premise and hypothesis were provided. This may suggest a mild hypothesis-only bias in the model. However, to confirm this, further investigation into the model's attention patterns during the decision-making process would be necessary.

Table 6.2: NLI accuracies

Test	Gemma 3 (1b)	Gemma 3 fine-tuned (1b)
Accuracy on SICK	0.44	0.58
Accuracy on SICK neutral examples	0.60	0.75
Accuracy on SICK neutral examples - hypernyms substitution	0.58	0.74
Accuracy on SICK - hypothesis only	0.47	0.53

6.4.1. Augmented dataset results on Gemma 3 and Gemma 3 fine-tuned

The improved synonym substitution solution's results on the Gemma 3 model are included in the table 6.3, along with the previous accuracy of the model on the original SICK dataset.

Table 6.3: NLI accuracies on SICK vs substituted SICK

Test	Gemma 3 (1b)	Gemma 3 fine-tuned (1b)
Accuracy on modified SICK	0.42	0.42
Accuracy on Neutral SICK - synonym substitution	**0.43**	**0.73**
Accuracy on Entailment SICK - synonym substitution	**0.24**	**0.42**
Accuracy on Contradiction SICK - synonym substitution	**0.74**	**0.26**
Accuracy on SICK	0.41	0.59
Accuracy on Neutral SICK	0.41	0.75
Accuracy on Entailment SICK	0.28	0.46
Accuracy on Contradiction SICK	0.70	0.25

The colors in the table 6.3 indicate whether the model's accuracy improved on the modified dataset or not: green indicating an increase in accuracy, while the color red indicating a decrease in accuracy.

The initial assumption and hypothesis for this experiment's output was that, when different words, although synonyms, are used for the subjects of the sentence pair, the model would have a harder time predicting the logical relationship between the two statements. The reason for this is that the model would have to rely on semantical understanding of the words instead of purely looking at word overlap to decide if the two concepts are the same in the premise and hypothesis, or if they are different, preventing the model from creating shortcuts in classification.

As we can observe there is an increase in accuracy over the neutral and contradiction examples, when the model was tested on the modified dataset, and there is an increase in accuracy when it was tested on entailment examples. The increase is 2% for the contradiction, 4% for the neutral and the decrease is 4% for the entailment examples.

These results align with the prediction, since it suggests that the model is more likely to predict contradiction or neutral when the subjects of the two sentences are different, and less likely to predict it as entailment. It suggests that this model might have limitations when it comes to fully understanding what a concept is referring to. Although, we have to note that the change in accuracy is not significant, therefore the model still maintains a certain robustness.

This experiment altogether raises some question marks about the model's semantical meaning understanding capabilities that would be worth investigating further in the future.

6.5. Results for different prompts on Gemma

Table 6.4 contains the accuracies of the original Gemma 3 instruct model (1b parameters) from LM Studio. This table compares the accuracies for different prompts that were given to the model. This test was only done for the entailment examples from the SICK dataset.

It is important to note that the model performed better (an increase of 20%), when the prompt explicitly asked the model to explain the reasoning, not only to analyze the relationship and then respond with one word.

Another interesting effect is that the model performed even better, when the prompt explicitly said that it was supposed to CLASSIFY the relationship between the two sentences.

Table 6.4: Prompt phrasing variations and their entailment accuracies on SICK

Prompt phrasing	**Accuracy – entailment – SICK**
”Analyse the relationship between these two sentences, **explain the reasoning and after explaining respond** with ONLY one word: - ’entailment’ if the hypothesis logically follows from the premise - ’contradiction’ if the hypothesis contradicts the premise - ’neutral’ if there’s no clear relationship”	0.24
“Analyse the relationship between these two sentences, and respond with ONLY one word: - ’entailment’ if the hypothesis logically follows from the premise - ’contradiction’ if the hypothesis contradicts the premise - ’neutral’ if there’s no clear relationship”	0.04
”Classify the relationship between a premise and hypothesis.” ”Respond with ONLY ONE WORD: - ’entailment’ if the hypothesis logically follows from the premise - ’contradiction’ if it contradicts the premise - ’neutral’ if there’s no clear relationship”	0.28

6.6. NLI results for different models compared

Table 6.5 presents the accuracy of several models evaluated on the SICK dataset for the NLI task. The Qwen3 model had the highest accuracy – 0.71 – and it outperforms both Gemma 3 and Llama 3.2, which achieve accuracies of 0.44 and 0.45, respectively.

What is interesting is the fact that Qwen3 is a much smaller model than the other models. It has 600 million parameters, while Gemma and Llama each have 1 billion. Even though it is almost half the size, Qwen3 still performs better. This suggests that Qwen3 is using its parameters more efficiently. Although it is important to note that the Llama and Gemma models used in this task are among the smaller variants of their respective model families.

However, Qwen3 is also much slower than the other models when generating responses. So, while it gives better results, it may not be the best choice for tasks where speed is important.

Table 6.5: NLI accuracies

Model name	No. of parameters	Accuracy
Gemma 3	1b	0.44
Llama 3.2	1b	0.45
Qwen 3	600m	0.71

6.7. Heatmap of Gemma

The heatmap in figure 6.4 represents the Gemma 3 (1b params) model's performance on NLI, on the SICK dataset. The overall accuracy measured is 0.44. We can notice here an interesting thing: the main faults of the model are when the true label is entailment, but the model predicts a neutral label (third row, middle square), and when the true label is neutral, but the model predicts it as contradiction. This suggests a "shifting" effect in judgement.

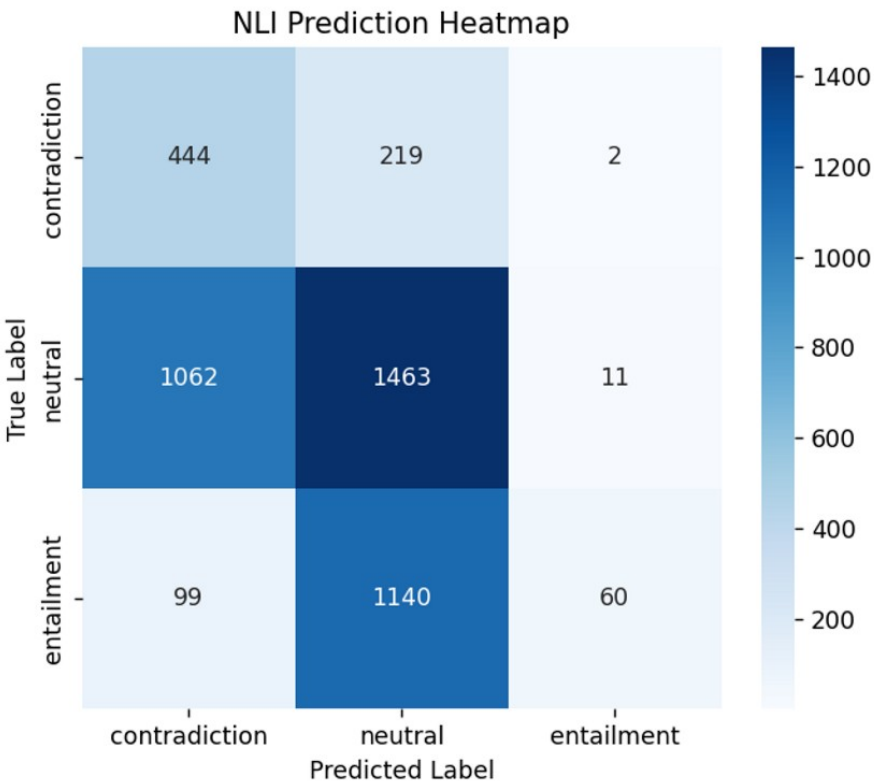

Figure 6.4: Gemma 3 results on SICK

In the figure 6.5 the gemma model's performance is shown, in an experiment where only the hypothesis was given to the model. Using the same prompt as before, an empty premise and the original hypothesis was given to the model, and the task was the same, to predict the label. It is interesting to notice, that in this experiment the model's accuracy was 0.47 compared to 0.44, when both the premise and hypothesis was given. However, the reason for this accuracy increase can come from the fact that in the SICK dataset the

majority of the examples are with the label neutral, and if and empty premise is given, then we can say that the model is rightfully assuming that the premise does not have any logical/entailment connection to the hypothesis.

Therefore, this increase of accuracy can be a simple coincidence and the byproduct of the SICK dataset's label distribution.

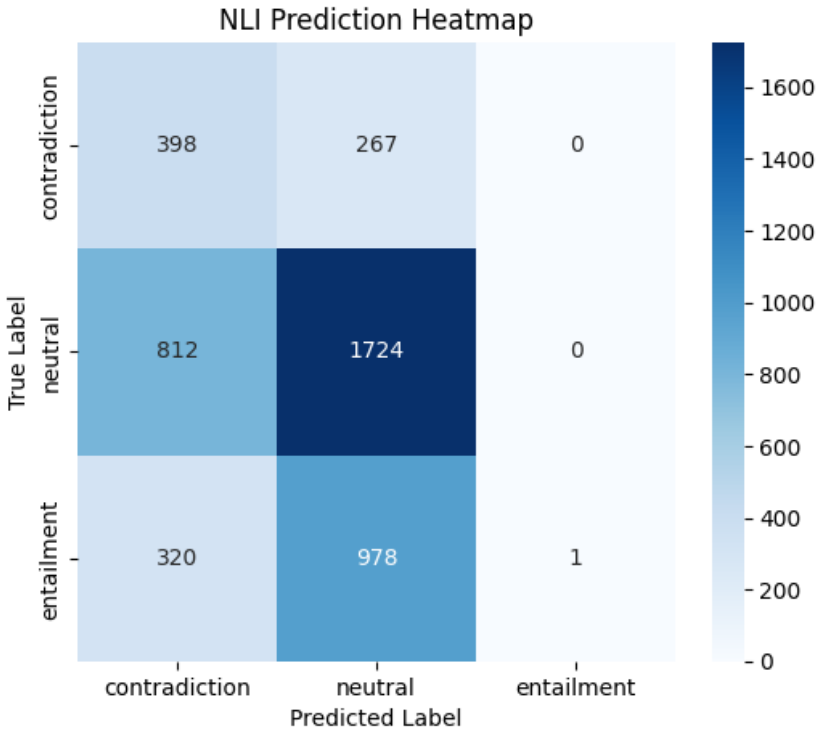

Figure 6.5: Gemma 3 model's results, when only the hypothesis is given

In the figure 6.6 the gemma model's performance is shown on the SICK dataset, substituted with synonyms for the subject of the premise sentences. In this heatmap, the same "shifting" effect can be observed as in the case of the Gemma base model's test on the original dataset, which could indicate that it is a side-effect of the model.

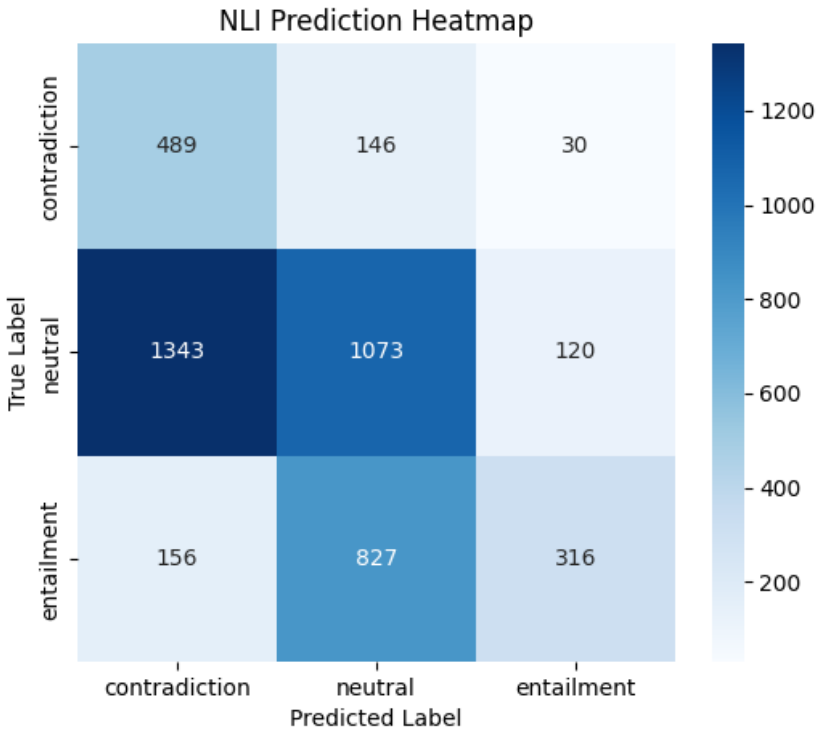

Figure 6.6: Gemma 3 model's results, for the modified dataset (synonym substitution)

Chapter 7. User's Manual

The resources of the project are available on GitHub. The repository that contains the code used to run in a Python environment on a local computer is the following: `https://github.com/deborasiko/siko_debora_thesis_local_codes.git`

There is an additional repository available on GitHub, where the fine-tuned model weights, the Google Colab and Kaggle notebooks are available. It can be accessed on the following link: `https://github.com/deborasiko/thesis_notebooks_model_weights.git`

7.1. Local environment

To be able to run the code from the GitHub repository, the user should first create a Python virtual environment. In this virtual environment, there are dependencies that should be installed. These dependencies are listed in the *requirements.txt* file, in the root folder of the first repository. The user can install all of the dependencies and libraries used, using the *pip install* command, once the environment is activated. To create the environment, the following command can be used:

```
    python -m venv <name_of_environment>
```

The code is structured in the following way: There are separate directories for different types of experiments: the *dataset_analysis_codes* directory contains the Python scripts used in the evaluation process of the SICK dataset, *the dataset_augmentation_codes* folder contains the Python scripts used in the development of the dataset modification. In the folder named *gemma-3-1b-it_test_codes* the code used for testing the gemma models (both the Gemma 3 base and the Gemma 3 fine-tuned versions) on NLI. Similarly, the folders named *llama_test_codes*, *qwen_test_codes*, *t5_test_codes* contains the code used to test the other models, respectively.

In the *gemma-3-1b-it-finetuned* folder the fine-tuned model's weights are stored. The *data_files* folder contains the datasets used in this project. It contains the base SICK dataset, as well as the entailment, contradiction and neutral subsets of the dataset. The modified datasets are also stored in this folder, as shown in figure 7.1:

```
data_files/
    augmented_ds_synonym.txt
    contradiction_sick_data.txt
    contradiction_sick_substituted_data.txt
    entailment_sick_data.csv
    entailment_sick_data.txt
    entailment_sick_substituted_data.txt
    found_synonyms.txt
    hypernyms.txt
    hyponyms.txt
    neutral_sd.txt
    neutral_sick_data.csv
    neutral_sick_data.txt
    neutral_sick_hypernyms_data.txt
    neutral_sick_hyponyms_data.txt
    neutral_sick_substituted_data.txt
    only_hypothesis.txt
    results_contradiction_candidate_synonyms_all.txt
    results_entailment_candidate_synonyms_all.txt
    results_neutral_candidate_synonyms_all.txt
    sick_neutral_data_synonym_sub.txt
    SICK_train.txt
```

Figure 7.1: Datasets in the data_files folder

To run the code, the user should first activate the virtual environment with the command

```
<name_of_environment>\Scripts\activate
```

,then navigate to the folders with the "*cd* ..." command and then with the

```
python <name_of_file>
```

command, the files can be run.

7.2. LM Studio models

To run the code for model testing, first the user should install LM Studio, and download the models used in this project. Then, to run a model in LM Studio, it needs to be loaded. This can be done by navigating to the developer page 7.2, then on the top, the user should select a model, from the list of the previously downloaded models, like shown in figure 7.3:

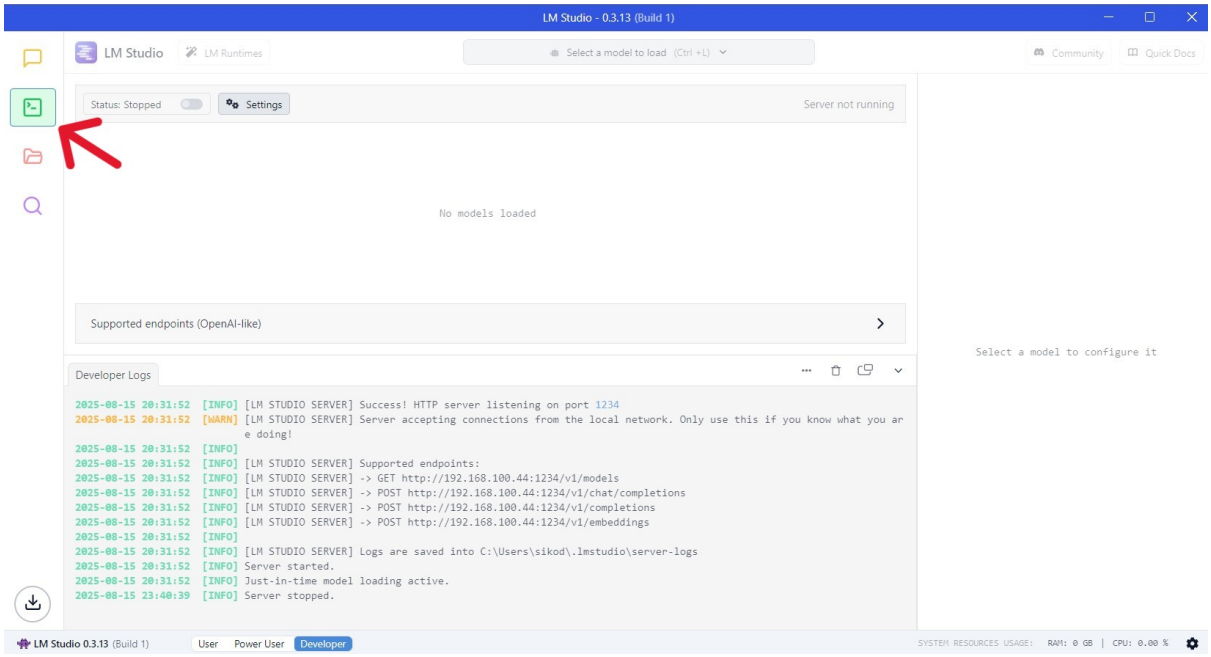

Figure 7.2: Step 1: Developer tool in LM Studio

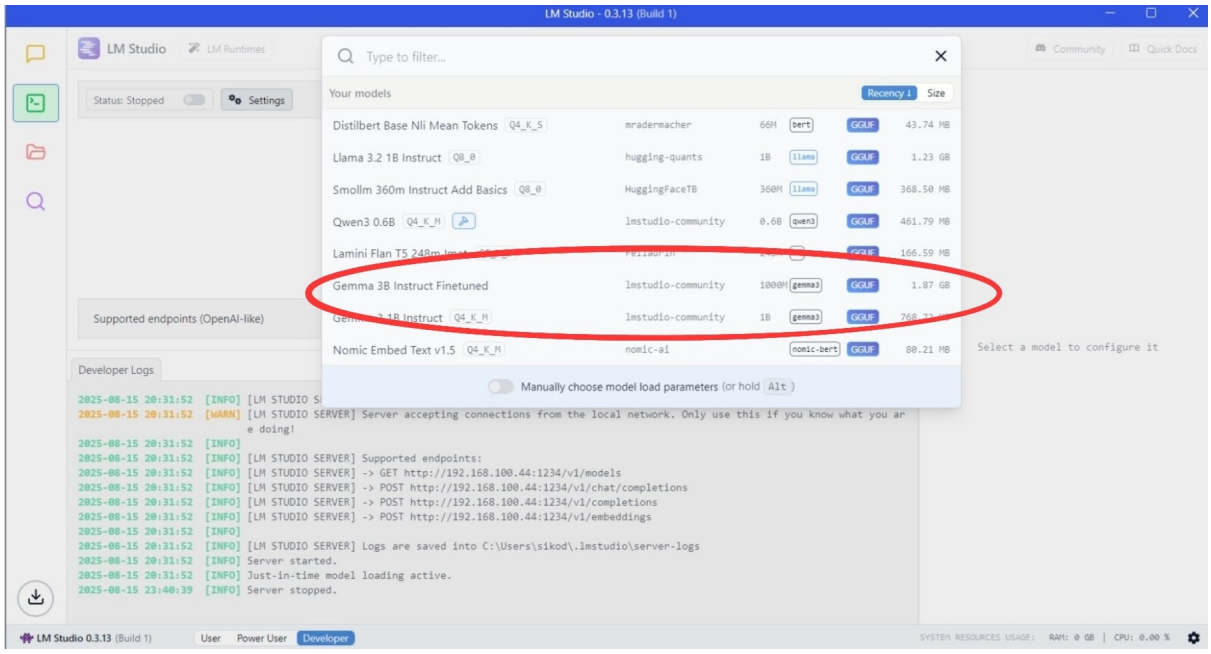

Figure 7.3: Step 1: Loading a model in LM Studio

After loading the chosen model, the user can run tests on it by running the Python scripts corresponding to this task. Note, that the status of the model that is loaded must be running.

Downloading a model is done by navigating to the search bar, and searching for a model by name.

7.3. Google Colab and Kaggle

To run the notebooks in Google Colab (for fine-tuning) or in Kaggle (dataset modification code) the notebooks should be taken from the second GitHub repository and they should be uploaded in the Google Colab or Kaggle working environment.

To run the code for the dataset augmentation in Kaggle, beside the notebook itself (by the name *synonym_substitution*), the necessary data files should be uploaded too (in the case of this experiment the files used are the distributed partial datasets for SICK, named *contradiction_sick_data.txt*, *entailment_sick_data.txt*, *neutral_sick_data.txt*, found in *data_files* directory). The datasets can be uploaded by navigating to the *Input* part on the right side of the Kaggle notebook, as it is shown in figure 7.4.

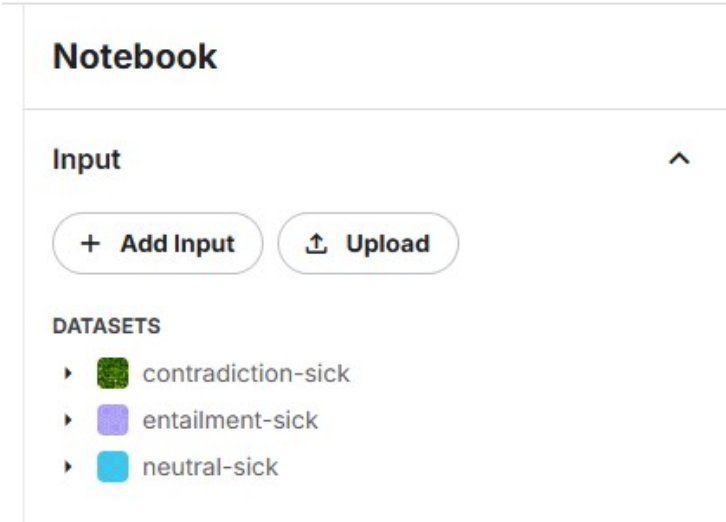

Figure 7.4: Kaggle inputs

7.4. fine-tuned model replication

To replicate the fine-tuning process, the user should choose one of the two options below:

1. Using the model weights
2. fine-tuning from scratch with Google Colab notebook

To execute the first option, all the user has to do is to is download the model weights from the repository, and clone the repository with the local solutions, then by executing the *merge_fine_tuned_model_with_base.py* script, the user will get the merged model. However, to transform it into the .GGUF format required by the LM Studio, an additional step must be taken.

First the user must set up Llama.cpp from the repository.

The scripts can be used by first cloning the *llama.cpp* repository:

```
git clone https://github.com/ggerganov/llama.cpp.git
```

Then the necessary libraries and dependencies should be installed:

```
pip install -r llama.cpp/requirements.txt
```

The second step is to merge the base model to the model weights, as defined previously. After merging, the user should navigate in the *llama.cpp* directory to the script which converts a model that is already on the local computer, into a .gguf format.

The user should run the command:

```
python llama.cpp/convert_hf_to_gguf.py <path_to_merged_model>
  --outfile <output_path>\<name_of_model>.gguf
  --outtype f16
```

The model in the gguf format will be stored in the directory and by the name scpecified by the *–outfile* parameter. The parameter *–outtype* defines the quantization type of the model, however, this is an optional parameter, if not defined it will default to f16, which means that the conversion outputs a model in float precision of 16.

The user should make sure that the merged model's directory is containing all the necessary files for conversion: *config.json*, *generation_config.json*,

<model_name>.safetensors, and *tokenizer.json*.

The converted .gguf model should be then uploaded to LM Studio and then it can be used. If the path specified by the *–outfile* parameter is the one pointing to where the LM Studio models are stored on the local computer, then the model will be visible in LM Studio too, and it can be loaded and used as defined in the previous sections.

Chapter 8. Conclusions

This thesis has explored the performance of small-scale text-generative transformer models on the task of Natural Language Inference (NLI). The work has been driven by four main objectives: analyzing how such models perform on the NLI task, analysis of the dataset used to perform NLI, fine-tuning a transformer model to compare its reasoning capability to its base version and experimenting with dataset augmentation through synonym substitution to explore further reasoning capabilities. This chapter presents a summary of the contributions, an analysis of the results and discusses possibilities for further improvement.

8.1. Personal Contributions and Critical Result Analysis

8.1.1. Analysis on NLI of smaller text-generative models

The first contribution of this research lies in the evaluation of smaller text-generative transformer models on NLI. These models were explored in a local environment via LM Studio. The analysis provided insights into the strengths and limitations of small-scale (less than 1b parameters) generative models, highlighting that there is still room for improvement for them to meet the state-of-the-art standards. The models explored in this research are: Gemma 3 model (with 1b parameters), Llama 3.2 (with 1b parameters) and Qwen 3 (with 600m parameters). The majority of the experiment was conducted on the Gemma 3 model, exploring also how it reacts to certain prompt changes, besides the NLI task.

The results of the first experiment suggest that some improvement is needed for these small-scale models to achieve a state-of-the-art performance in logical reasoning. There can be a trade-off between speed and accuracy, suggested by the fact that the Qwen 3 model (which is smaller than the other models in the experiment) is more accurate (70% for NLI) than the Llama 3.2 (45%) or the Gemma 3 (44%) models, but at the same time it takes a significantly longer time to generate the response. The different prompting results for the Gemma 3 model highlight the assumption that proper prompting is important when using text-generative models, while raising the question whether this indicates a lack of robustness and understanding in the models.

8.1.2. Dataset Analysis

The second contribution involved the analysis of the SICK dataset used for the NLI task. The analysis was conducted to insure the dataset is artifact free and a suitable metric for NLI evaluation. Firstly, this analysis included sentence length distribution comparisons across the labels, to make sure that the dataset is free from the artifact of unequal length distribution. Secondly, it contained an analysis on Jaccard Similarity, to

compare the lexical similarity of the sentence pairs.

The analysis of the dataset suggests that on the surface level the SICK dataset is artifact free, regarding the sentence length distribution and Jaccard Similarity. The results show that there is no particular label, where the sentence length difference would be too large, compared to other labels. Regarding Jaccard Similarity, the histogram suggests that the lexical overlap for the neutral labels are lower than for entailment and contradiction labels, which could indicate a potential artifact in the dataset. However, the Jaccard Similarity provides insight only into the actual word overlap between the sentences, the analysis of whether two sentences refer to same topics semantically, requires more in-depth analysis of the sentence meanings.

8.1.3. Fine-tuning and Evaluation

The third contribution, meeting objective 3, involved fine-tuning a model specifically on an NLI dataset. The first part of the contribution was identifying the appropriate method for fine-tuning along with the model to fine-tune, which was Gemma 3 for the model and PEFT for the method, in this case. PEFT was chosen because it provides a fine-tuning method that is both effective and time efficient. The fine-tuning itself was achieved by utilizing LoRA, and merging the new weights and the Gemma 3 base model. Later this fine-tuned model was evaluated on the same NLI task as the Gemma 3 base itself, comparing their results.

The analysis provided insights into the strengths and limitations of small-scale generative models, highlighting that while the fine-tuning itself increased the overall accuracy of the model, the nuanced mistakes of the model remained. This indicates that there is a greater underlying issue worth investigating.

The results of this experiment suggest that, while the fine-tuned model's overall accuracy is higher, this is likely due to the fact that it seems to generalize the majority of the examples towards the neutral label. Seemingly, it shows an effect of having stricter rules and thresholds for an example to be considered contradiction or entailment, majority of the cases being predicted as neutral. While the Gemma 3 base model's predictions seem to be biased towards the contradiction label, the Gemma 3 fine-tuned version's predictions are leaning towards the neutral label. However, this experiment's results should be analyzed with caution, since the Gemma 3 fine-tuned version's bias could be caused by overfitting during the fine-tuning process, or due to learned biases in SNLI dataset (which it was fine-tuned on).

8.1.4. Dataset Augmentation and Interpretation of Results

The fourth contribution of this thesis concerned dataset augmentation to later test the models on the augmented dataset. By enriching the dataset with lexical variations, the intention was to gain a better insight on how the Gemma 3 model utilizes NLU in the decision making process of the NLI task. The SICK dataset was first modified by eliminating the premises altogether, creating a separate dataset to analyze the Gemma 3 model on hypothesis-only bias. Secondly, the dataset was augmented with creating a new dataset, where the subjects of each premise were substituted with their hypernyms. Thirdly, an additional dataset was created where instead of hypernyms, synonym substitution was used. For the augmentation process POS tagging and WordNet were utilized. The Gemma 3 and the Gemma 3 fine-tuned versions were evaluated in these datasets, gaining a better insight into how the models understand the written text.

While the augmentation made the dataset more diverse, in some cases it also introduced certain semantic ambiguities that made the sentence pair's relationship questionable.

The hypernym substitution experiment showed one of these limits clearly. Because replacing a subject with a more general word can easily break the entailment relationship. This type of augmentation was mainly meaningful for neutral examples. The model's accuracy did not change much here, which suggests that it can handle some level of abstraction. However, it also shows that increasing generality in the premises makes it harder for the model to understand fine logical distinctions.

The synonym substitution experiment revealed more about the model's potential biases. The results showed that the accuracy improved for neutral and contradiction cases, but dropped for entailments. This pattern suggests that Gemma 3 might rely on word overlap to decide whether two sentences entail each other. When overlap is reduced by synonym substitution, the model fails to recognize that the meaning is still the same. At the same time, the reduced overlap makes it easier for the model to separate neutral and contradiction cases. This suggests that the model's understanding is surface level, using cues like word matching to predict the labels.

The fact that the fine-tuned version did not always improve results also shows that fine-tuning may have amplified dataset-specific patterns instead of helping the model reason more clearly.

The experiment also highlights the challenges of dataset augmentation with synonyms. The reason being that augmentation itself can introduce problems. Using WordNet sometimes produced biased or contextually wrong synonyms, which means the dataset needs careful filtering to avoid misleading results.

8.2. Possibilities of Future Improvements

Looking forward, several paths for improvement and further research can be identified. Firstly, the experiments using dataset augmentation should be repeated for the other models in the thesis (Qwen 3 and Llama 3.2), to gain a better insight into how specific the results are to Gemma 3.

Secondly, the dataset analysis can be more in-depth when it comes to topic overlap and topic similarity, to gain a better insight into the dataset's potential biases. For this purpose, Cosine Similarity can be used to detect the similarity of the sentence embeddings in the premise-hypothesis pairs. An investigation on the frequency of certain words across the labels would also provide a powerful insight into potential artifacts when it comes to preferable wording for a specific label.

The fine-tuning process showed that by teaching the model domain-specific knowledge, small-scale generative transformer models can achieve meaningful gains in NLI performance. However, the fine-tuning process could be replicated on a higher scale with more resources, and it should be performed on the Llama 3.2 and Qwen 3 models too.

More advanced augmentation methods, such as paraphrase generation or adversarial example construction, could provide richer and more reliable training data than synonym substitution alone. The WordNet based approach could benefit from even more precise evaluation and substitution, while the POS tagging algorithm can be improved to include grouped subjects as well. The augmentation process could be expanded to the hypotheses as well.

Another insightful experiment would be investigating the model's attention weights, when it comes to decision making to gain a better insight into what the models pay attention to at each step and ultimately how do the text-generative models conduct the reasoning.

In conclusion, this thesis shows that small-scale LLMs still need to improve in logical reasoning compared to state-of-the-art models, their reliance on surface-level patterns over true semantic reasoning highlights both their current limitations and the need for more robust evaluation and augmentation strategies.

Bibliography

[1] I. Schlag, P. Smolensky, R. Fernandez, N. Jojic, J. Schmidhuber, and J. Gao, "Enhancing the transformer with explicit relational encoding for math problem solving," *arXiv preprint arXiv:1910.06611*, 2019.

[2] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models," *arXiv preprint arXiv:2410.05229*, 2024.

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[4] A. Saparov and H. He, "Language models are greedy reasoners: A systematic formal analysis of chain-of-thought," *arXiv preprint arXiv:2210.01240*, 2022.

[5] O.-M. Camburu, T. Rocktäschel, T. Lukasiewicz, and P. Blunsom, "e-snli: Natural language inference with natural language explanations," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[6] A. Baas, "Mitigating bias in the snli dataset," B.S. thesis, 2021.

[7] V. A. Negru, R. Vacareanu, C. Lemnaru, M. Surdeanu, and R. Potolea, "Morphnli: A stepwise approach to natural language inference using text morphing," *arXiv preprint arXiv:2502.09567*, 2025.

[8] X. Feng, Z. Wan, M. Wen, S. M. McAleer, Y. Wen, W. Zhang, and J. Wang, "Alphazero-like tree-search can guide large language model decoding and training," *arXiv preprint arXiv:2309.17179*, 2023.

[9] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," *arXiv preprint arXiv:1508.05326*, 2015.

[10] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. R. Bowman, and N. A. Smith, "Annotation artifacts in natural language inference data," *arXiv preprint arXiv:1803.02324*, 2018.

[11] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," *arXiv preprint arXiv:1704.05426*, 2017.

[12] M. Marelli, L. Bentivogli, M. Baroni, R. Bernardi, S. Menini, and R. Zamparelli, "Semeval-2014 task 1: Evaluation of compositional distributional semantic models on full sentences through semantic relatedness and textual entailment," in *Proceedings of the 8th international workshop on semantic evaluation (SemEval 2014)*, 2014, pp. 1–8.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[14] M. Surdeanu and M. A. Valenzuela-Escárcega, *Deep learning for natural language processing: a gentle introduction.* Cambridge University Press, 2024.

[15] K. Diederik, "Adam: A method for stochastic optimization," *(No Title)*, 2014.

[16] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization." *Journal of machine learning research*, vol. 12, no. 7, 2011.

[17] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models." *ICLR*, vol. 1, no. 2, p. 3, 2022.

[18] G. A. Miller, "WordNet: A lexical database for English," in *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*, 1994. [Online]. Available: https://aclanthology.org/H94-1111/

[19] H. Face. (2025) Lora, in *PEFT (Parameter-Efficient Fine-Tuning) package reference*. Documentation for version v0.13.0. [Online]. Available: https://huggingface.co/docs/peft/v0.13.0/en/package_reference/lora